



武汉大学
WUHAN UNIVERSITY

S-OS (SuperOS)

内核设计文档

成员：包一帆，李嘉乐，庄廷钊

指导教师：蔡朝晖

2026 年 6 月

目录

第一章 概述	2
1.1 系统架构总图	2
1.2 项目定位	3
1.3 代码组织	3
第二章 进程与线程管理	4
2.1 任务模型：三层结构	4
2.1.1 proc_t: 资源容器的组织	4
2.1.2 thread_t: 执行现场的组织	5
2.1.3 cpu_t: 最小化调度现场	5
2.2 进程创建路径对比	5
2.2.1 fork() 的完整复制路径	6
2.2.2 clone() 的完整复制路径	6
2.2.3 clone_thread() 的共享页表机制	6
2.3 调度器与上下文切换	7
2.3.1 scheduler(): 二级扫描逻辑	7
2.3.2 时间片轮转与超时检测	8
2.4 阻塞、唤醒与同步路径	8
2.4.1 基于 channel 的睡眠模型	8
2.4.2 Futex: 线程级等待与唤醒	9
2.5 信号与定时器	10
2.5.1 信号状态的组织	10
2.5.2 信号发送与终止标记的协作	10
2.6 退出、等待与资源回收	10
2.6.1 exit -> reparent -> ZOMBIE -> wait -> freeproc 回收链	10
2.6.2 freeproc() 的资源释放顺序	11
2.6.3 initproc 对孤儿进程的回收	11
2.7 当前设计的边界与局限	11
第三章 内存管理	13
3.1 总体架构	13
3.2 Buddy System	13
3.2.1 数据结构	13
3.2.2 buddy_init(): 自举过程	14

3.2.3	buddy_alloc(): 分配算法	14
3.2.4	buddy_free(): 释放与合并算法	15
3.2.5	PAGE_NUM 的双架构差异	15
3.2.6	页级引用计数	16
3.3	SLAB 分配器	16
3.3.1	固定 size class 设计	16
3.3.2	slab_alloc(): 分配流程	16
3.3.3	自举过程: simple_alloc -> slab_init	17
3.4	页表与地址映射	17
3.4.1	walk(): 多级页表遍历核心算法	17
3.4.2	RISC-V 与 LoongArch 的页表差异	18
3.5	VMA 与用户地址空间	18
3.5.1	地址空间布局	18
3.5.2	VMA 链表与区域语义	19
3.5.3	alloc_vma / alloc_mmap_vma 的地址选择策略	19
3.5.4	mmap / munmap 的实现流程	20
3.5.5	fork() 时 VMA 的共享与私有处理	20
3.5.6	pagefault_handler 的决策流程	20
3.6	小结	21
第四章	硬件抽象层与异常处理机制	22
4.1	分层架构与代码边界	22
4.2	双架构启动链	22
4.2.1	RISC-V 启动链	22
4.2.2	LoongArch 启动链	23
4.3	Trapframe 与 Context 的分工	23
4.4	用户态 Trap 主路径	24
4.4.1	usertrap() 异常分类	25
4.4.2	devintr() 中断分发	26
4.4.3	缺页处理	27
4.5	内核态 Trap	27
4.6	LoongArch TLB Refill	28
4.7	hsai_usertrapret() 返回路径	29
第五章	系统调用与用户态接口	31
5.1	设计目标	31
5.2	ABI 入口与用户态调用桩	32
5.2.1	双架构调用约定	32

5.2.2	GEN_USER_SYSCALL 宏	32
5.3	从 Trap 到 syscall() 的统一分发	33
5.3.1	syscall() 分发器	33
5.4	参数搬运机制	33
5.5	系统调用分类	34
5.6	兼容层设计	35
5.6.1	BusyBox 虚拟文件系统	35
5.6.2	LD_LIBRARY_PATH 注入	36
5.6.3	/dev/shm 重映射	36
5.6.4	Synthetic Mount 机制	37
5.7	本章小结	37
第六章	文件系统与程序装载	38
6.1	总体架构	38
6.2	文件对象层	39
6.2.1	struct file 的类型系统	39
6.2.2	生命周期管理	40
6.2.3	读写类型分发	40
6.2.4	FILE_OPS 分发表	40
6.3	EXT4 适配层	41
6.3.1	打开路径: vfs_ext4_openat	41
6.3.2	读写路径: 分页中转缓冲区	41
6.3.3	目录遍历	41
6.3.4	元数据访问	42
6.3.5	删除操作中的 inode 引用协调	42
6.4	块缓存层	42
6.4.1	核心数据结构	42
6.4.2	四个核心接口	42
6.4.3	bget 的 LRU 回收策略	42
6.4.4	blockdev.c 的 EXT4 适配	43
6.5	管道实现	43
6.6	BusyBox 虚拟文件	44
6.6.1	虚拟文件列表	44
6.6.2	按路径分发	44
6.7	execve 装载器	45
6.7.1	执行流程	45
6.7.2	PT_LOAD 段装载与 loadseg	45
6.7.3	PIE 支持	45

6.7.4	动态解释器处理	45
6.7.5	辅助向量构造	46
6.7.6	脚本兼容与重定向	46
6.8	本章小结	47
第七章	驱动层设计与实现	48
7.1	概述	48
7.2	控制台子系统	48
7.2.1	读写分离架构	48
7.2.2	行规约处理	48
7.2.3	UART 发送环形缓冲	49
7.2.4	字符设备注册	49
7.3	定时器	49
7.3.1	双架构定时器初始化对比	49
7.3.2	事件链: timer_tick 到 SIGALRM	50
7.4	RISC-V PLIC	51
7.5	RISC-V VirtIO MMIO 块设备	52
7.5.1	virtio_disk_init 状态机	52
7.5.2	三描述符链与请求提交	52
7.5.3	中断回收	53
7.5.4	描述符表内存布局	53
7.6	LoongArch PCI 与 VirtIO-PCI 块设备	53
7.6.1	PCI 设备枚举	53
7.6.2	Capability 解析	54
7.6.3	轮询完成路径	54
7.6.4	两种架构的差异对比	55
7.7	驱动与上层接口	55
7.8	本章小结	56
第八章	总结与展望	57
8.1	设计总结	57
8.2	后续优化方向	57

第一章 概述

1.1 系统架构总图

系统采用自底向上的分层结构，层间依赖关系如图 1.1 所示。SOS 内核在统一代码基线上同时支持 RISC-V 64 和 LoongArch 64 两种体系结构，图中以体系结构相关与体系结构无关两类模块展示了这一设计。

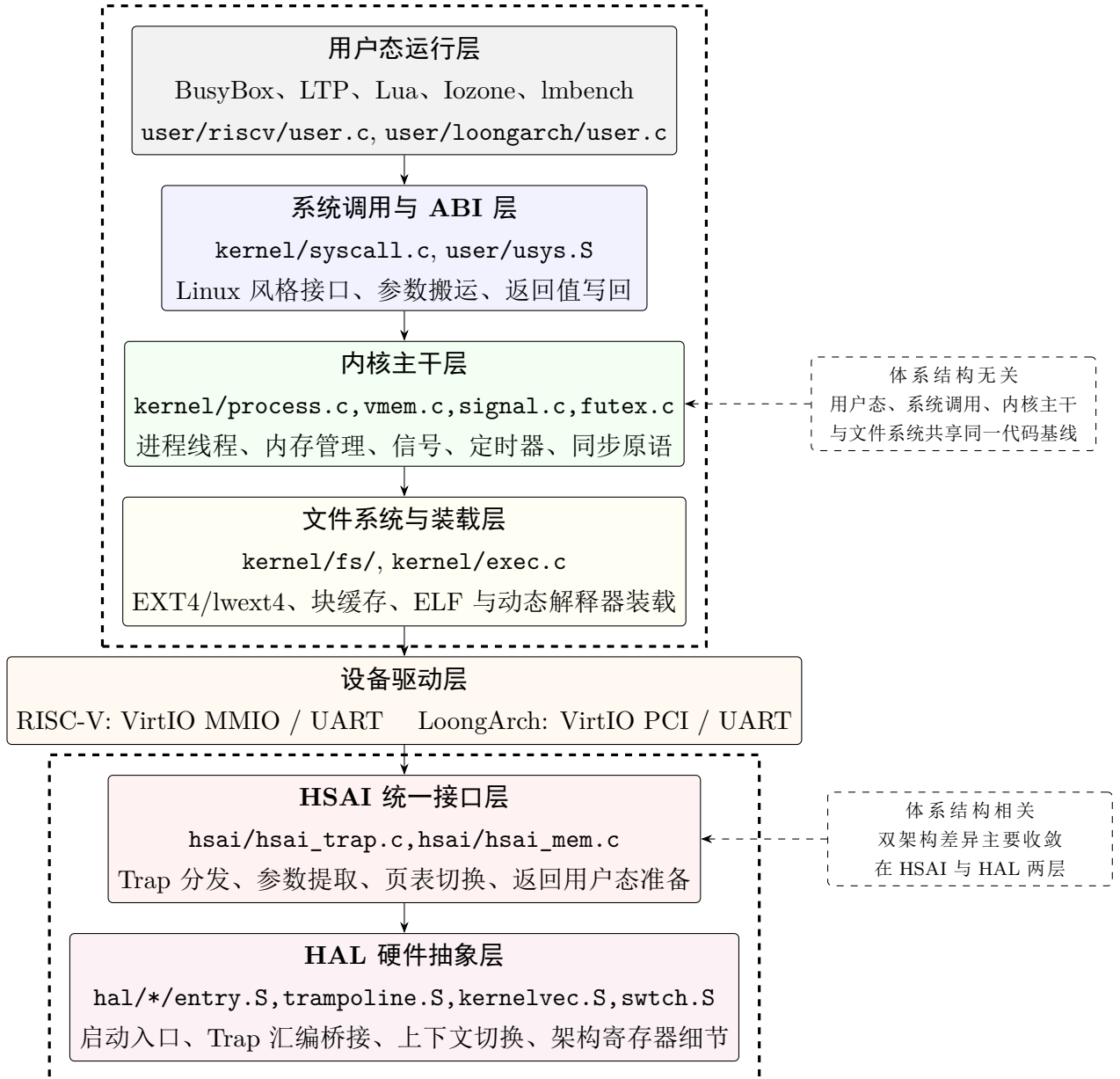


图 1.1: SOS 内核全架构分层图

1.2 项目定位

SOS (SuperOS) 是一个宏内核操作系统，面向 RISC-V 64 和 LoongArch 64 两种体系结构，覆盖进程与线程管理、虚拟内存管理（Buddy + Slab + 页表 + VMA）、EXT4 文件系统（集成 lwext4）、VirtIO 块设备驱动（MMIO 与 PCI）以及 Linux 风格系统调用接口（进程、内存、文件、定时器、事件、socket 类别），可运行 basic、buxybox、libctest、libcbench、lua、部分 ltp 等测试用例。

1.3 代码组织

各目录的职责和代表性文件见表 1.1。

表 1.1: 目录结构与职责

目录	职责	代表性文件
hal/	架构入口、寄存器现场保存恢复、上下文切换、UART	entry.S, trampoline.S, swtch.S, uart.c
hsai/	Trap 分发、页表硬件配置、PLIC、内核打印	hsai_trap.c, hsai_mem.c, plic.c, print.c
kernel/	进程线程、内存管理、系统调用、信号、定时器	process.c, vmem.c, syscall.c, signal.c, futex.c
kernel/fs/	文件对象、路径处理、块缓存、EXT4 适配	vfs_ext4.c, file.c, bio.c, blockdev.c
kernel/driver/	块设备驱动（VirtIO MMIO / PCI）	virt.c, pci.c, virtio_disk.c
kernel/exec.c	ELF 与脚本装载	exec.c
user/	首个用户态程序、系统调用桩、测试框架	user.c, usys.S, usys_macro.inc
include/	内核核心头文件、HAL/HSAIL 接口、文件系统定义	process.h, thread.h, vmem.h, vma.h

架构依赖关系为：hal/ → hsai/ → kernel/（核心层与文件系统层）。其中，hal/ 和 hsai/ 主要容纳架构相关代码，kernel/ 主体保持双架构共享。驱动层在 kernel/driver/ 内部按架构分目录，例如 kernel/driver/riscv/ 和 kernel/driver/loongarch/。

第二章 进程与线程管理

2.1 任务模型：三层结构

当前内核将执行实体拆分为三个层次：进程（`proc_t`）持有资源，线程（`thread_t`）持有执行现场，CPU 本地状态（`cpu_t`）保存调度器切换现场。三层结构的核心动机是：资源不随执行流切换而复制，执行现场不因进程创建而膨胀，调度器不关心上层语义。

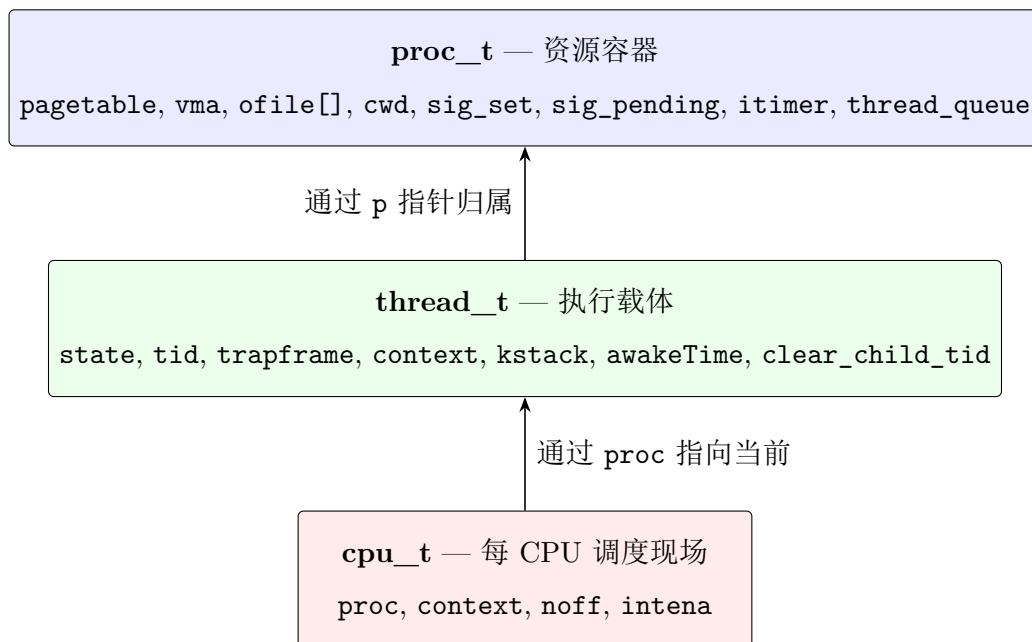


图 2.1: 进程/线程/CPU 的三层关系

2.1.1 `proc_t`: 资源容器的组织

`proc_t` 定义在 `include/kernel/process.h`。其成员按职能可分为五组：

职能组	关键字段	用途
地址空间	<code>pagetable</code> , <code>vma</code> , <code>sz</code> , <code>virt_addr</code>	持有用户页表指针、VMA 链表头、低地址连续区大小和程序入口偏移
文件系统	<code>ofile[NFILE]</code> , <code>cwd</code> , <code>ofn</code>	打开文件描述符表、当前工作目录、文件描述符数量限制
信号	<code>sig_set</code> , <code>sigact</code> , <code>ion[]</code> , <code>sig_pending</code>	信号阻塞掩码、每个信号的处理函数、挂起信号位图

定时器	<code>itimer, alarm_ticks, timer_active</code>	区间定时器值 (<code>setitimer</code> 接口)、时钟到期计数、定时器激活标记
线程管理	<code>thread_queue, main_thread, thread_count, thread_ad_num</code>	线程链表头、“当前线程”指针、线程计数

表 2.2: `proc_t` 关键成员分组

2.1.2 `thread_t`: 执行现场的组织

`thread_t` 定义在 `include/kernel/thread.h`, 集中保存执行流自身的状态:

- **调度状态:** `state` (枚举 `t_UNUSED` `t_TIMING`)、`chan` (等待 channel)、`awakeTime` (定时唤醒截止时间)。
- **执行现场:** `trapframe` (Trap 返回用寄存器镜像)、`context` (调度切换用 callee-saved 寄存器)。两者分工明确——`trapframe` 在用户态与内核态之间切换时保存/恢复全部寄存器, `context` 仅保存 `ra/sp/s0-s11`。
- **内核栈:** `kstack` 与 `kstack_pa`。主线程复用进程的 `kstack`, 附加线程由 `clone_thread()` 单独分配一页物理页作为内核栈, 并映射到内核页表。
- **链表节点:** `elem` 用于挂入 `proc_t` 的 `thread_queue`, 形成进程内的线程集合。

2.1.3 `cpu_t`: 最小化调度现场

`cpu_t` 定义在 `include/kernel/cpu.h`。当前 NCPU 设为 1, 但该结构体已明确划分每 CPU 状态与进程状态的边界: `proc` 指向当前运行进程, `context` 保存调度器寄存器现场, `noff` 与 `intena` 用于中断嵌套计数。切换时 `hsai_swch()` 仅在 `cpu->context` 与 `p->context` 之间切换, 不涉及地址空间或文件系统等高层状态。

2.2 进程创建路径对比

维度	<code>fork()</code>	<code>clone()</code>	<code>clone_thread()</code>
页表策略	<code>uvmcopy()</code> 逐页复制内容	<code>uvmcopy()</code> 逐页复制内容	共享父进程页表 (无复制)
VMA 策略	<code>vma_copy()</code> + <code>vma_map()</code> 深拷贝 (私有页复制, 共享页引用计数)	同 <code>fork()</code>	继承父进程 <code>thread_queue</code> , 不复制 VMA
返回值	父进程返回子进程 PID, 子进程返回 0	子进程 PID	新线程 TID
TLS 处理	不涉及	不涉及	<code>CLONE_SETTLS</code> 设置 <code>tp</code> 寄存器

栈布局	复制父进程栈	用户提供 <code>stack + args_t</code>	用户提供 <code>stack_va + 新内核栈</code>
文件描述符	<code>dup()</code> 全部打开文件	<code>dup()</code> 全部打开文件	共享（不复制）

表 2.4: 三种创建路径的对比

2.2.1 `fork()` 的完整复制路径

`fork()` 对应“完整复制进程”的语义：新建子进程对象，复制父进程的大部分可见运行状态，同时给予进程分配独立的地址空间与主线程现场。其关键步骤可概括为：

1. 调用 `allocproc()` 创建 `proc_t`，完成 PID、内核栈、活动 `trapframe` 等基础资源初始化。
2. 调用 `uvmcopy()` 复制父进程用户页，为子进程建立独立页表与物理页副本。
3. 调用 `vma_copy()` 复制 VMA，共享映射保留共享关系，私有映射维持父子地址空间分离。
4. 通过 `dup()` 复制文件描述符，继承工作目录、身份权限、信号与定时器等进程属性。
5. 复制执行现场至子进程 `trapframe`，子进程返回值置 0，父进程返回子 PID。
6. 子进程设为可运行并入调度队列，父子调度独立，仅共享显式定义的内核对象。

2.2.2 `clone()` 的完整复制路径

`sys_clone()` (`kernel/syscall.c`) 承担创建语义的总分发：

```
sys_clone(flags, stack, ptid, tls, ctid):
    if stack == 0 -> fork() // 完整复制进程
    if flags & CLONE_VM -> clone_thread() // 共享地址空间，创建线程
    else -> clone() // 独立页表，新进程
```

2.2.3 `clone_thread()` 的共享页表机制

`clone_thread()` (`kernel/process.c`) 不调用 `uvmcopy()`，新线程直接使用父进程的 `p->pagetable`。其步骤为：

1. 调用 `alloc_thread()` 分配 `thread_t`，该线程的 `trapframe` 是新分配的物理页，内容通过 `copytrapframe()` 从父进程 `p->trapframe` 复制。
2. 分配一页物理页作为新线程的内核栈，并通过 `mappages()` 映射到内核页表（虚拟地址为 `p->kstack - PGSIZE * p->thread_num`）。
3. 从用户空间复制 `args_t`（含入口函数地址、参数、控制信息），设置新线程的 `trapframe->epc/era` 为用户入口，`sp` 为用户提供的栈指针。
4. 处理 `CLONE_PARENT_SETTID`、`CLONE_CHILD_SETTID`、`CLONE_CHILD_CLEARTID` 等标志，将线程 ID 写入指定用户地址。
5. 设置 `t->state = t_RUNNABLE`，挂入父进程的 `thread_queue`。

当前实现中仍保留 `exit(0)` 占位调用，线程创建路径尚未完全收束。

2.3 调度器与上下文切换

2.3.1 `scheduler()`: 二级扫描逻辑

调度器主循环实现在 `kernel/process.c`，扫描顺序为先选进程，再选线程。

Algorithm 2.1: `scheduler()` 主循环算法

```

1: while true do
2:   for all 进程池中的进程 p do
3:     if p 未处于 RUNNABLE then
4:       continue
5:     end if
6:     for all p 的 thread_queue 中的线程 t do
7:       if t 处于 t_RUNNABLE then
8:         选择 t 作为本轮执行线程
9:       else if t 处于 t_TIMING 且 awakeTime 已到期 then
10:        将 t 视为可运行线程并参与本轮调度
11:       else
12:        continue
13:       end if
14:       调用 futex_clear(p->main_thread) 清理残留等待项
15:       令 p->main_thread = t
16:       复制线程私有 context 和 trapframe 到进程活动镜像
17:       设置 p->state = RUNNING, t->state = t_RUNNING
18:       调用 hsai_swch() 进入该进程
19:       从 hsai_swch() 返回后保存线程现场
20:       if t 未成为 t_ZOMBIE then
21:         将 t 移至线程队列队尾
22:       end if
23:       break
24:     end for
25:     释放 p 的锁并继续扫描下一进程
26:   end for
27: end while

```

线程可运行性的判定包括两个条件：

- 显式处于 `t_RUNNABLE` 状态；或

- 处于 `t_TIMING` 状态且 `awakeTime` 已到期 (`awakeTime < r_time() + (1LL << 35)`), 右侧偏移量用于防止 `awakeTime` 为负导致的误判)。

选中线程后的现场切换涉及三次复制:

1. `p->main_thread = t`: 将进程的”当前线程”指针指向被选中线程。
2. `copycontext(&p->context, &p->main_thread->context)`: 将线程私有上下文复制到进程活动镜像。
3. `copytrapframe(p->trapframe, p->main_thread->trapframe)`: 将线程私有 Trap 帧复制到进程活动镜像。

切换前还调用 `futex_clear(p->main_thread)`, 清理该线程在 `futex` 等待队列中的残留项。

2.3.2 时间片轮转与超时检测

当前调度器不维护单独的每线程时间片计数器。用户态时钟中断在 `hsai/hsai_trap.c` 中被捕获后, 调用 `yield()` 退回调度器, 同时调用 `timer_tick()` 更新全局 `ticks` 并遍历进程池检查定时器到期。超时线程的唤醒由 `scheduler()` 中的 `t_TIMING` 判定逻辑承担: `futex` 等待中设置了 `awakeTime` 的线程, 在调度器扫描时被自动发现并改为可运行。

2.4 阻塞、唤醒与同步路径

2.4.1 基于 channel 的睡眠模型

`sleep_on_chan()` 与 `wakeup()` 构成了内核内部最常用的阻塞原语。

`sleep_on_chan()` 的核心逻辑:

```
sleep_on_chan(chan, lk):
    if lk != &p->lock:
        acquire(&p->lock) // 防止错过 wakeup
        release(lk) // 释放调用者的锁
    p->chan = chan
    p->state = SLEEPING
    p->main_thread->state = t_RUNNABLE // 线程保持在就绪态
    sched()
    p->chan = 0
    if lk != &p->lock:
        release(&p->lock)
        acquire(lk)
```

关键设计决策: 将 `p->main_thread->state` 设置为 `t_RUNNABLE` 而非 `t_SLEEPING`。这意味着 channel 睡眠机制本质上是**进程级阻塞**——进程状态变为 `SLEEPING`, 但线程保留就绪状态。这是因为当前实现中每个进程在 channel 睡眠时只存在一个活跃线程, 没有必要将线程也标记为阻塞。

wakeup() 与之对应，只扫描进程池：

```
wakeup(chan):
    for p in pool:
        if p->state == SLEEPING && p->chan == chan:
            p->state = RUNNABLE
```

2.4.2 Futex: 线程级等待与唤醒

Futex 子系统实现在 kernel/futex.c。其核心数据结构是一个线性数组：

```
typedef struct futex_queue {
    uint64 addr; // futex 用户空间地址
    thread_t *thread; // 在该地址上等待的线程
    uint8 valid; // 队列项是否有效
} futex_queue_t;

futex_queue_t futex_queue[FUTEX_COUNT]; // FUTEX_COUNT = 2048
```

futex_wait() 的查找与等待流程：

Algorithm 2.2: futex_wait() 队列查找算法

- 1: 在 futex_queue[0..FUTEX_COUNT) 中查找首个 valid == 0 的空项
- 2: if 未找到空项 then
- 3: panic
- 4: end if
- 5: 在该队列项中写入 addr、等待线程指针和 valid = 1
- 6: if ts 为 NULL then
- 7: 设置 th->state = t_SLEEPING
- 8: else
- 9: 将超时时间折算写入 th->awakeTime
- 10: 设置 th->state = t_TIMING
- 11: end if
- 12: 设置 th->p->state = RUNNABLE
- 13: 调用 sched() 主动让出 CPU

与 channel 睡眠的关键区别：futex 等待将线程状态置为不可运行（t_SLEEPING 或 t_TIMING），同时将进程状态设回 RUNNABLE。这使得调度器可以在该进程的其他线程之间切换。

futex_wake() 扫描队列，将所有匹配地址的有效项对应线程的状态改回 t_RUNNABLE。futex_clear() 在线程被调度器选中时调用，用于清除该线程在队列中的所

有残留项。

2.5 信号与定时器

2.5.1 信号状态的组织

信号相关状态全部存放在 `proc_t` 中：

- `sigaction[SIGRTMAX+1]`：处理函数配置，通过 `sys_rt_sigaction()` 设置。
- `sig_set`：信号阻塞掩码，通过 `sigprocmask()` 修改。实现中强制 `SIGKILL`、`SIGTERM` 和 `SIGSTOP` 不可被阻塞（`kernel/signal.c`）。
- `sig_pending`：挂起信号位图，由 `kill()`、`tgkill()`、`timer_tick()` 写入。
- `killed`：终止标记，当终止类信号未安装处理函数时同步设置。

2.5.2 信号发送与终止标记的协作

`kill()` 的执行逻辑：

```
kill(pid, sig):
    for p in pool:
        if p->pid == pid:
            p->sig_pending.__val[0] |= (1UL << sig)
            if signal_should_terminate(sig) AND
                p->sigaction[sig].sa_handler == NULL AND
                (p->killed == 0 || p->killed > sig):
                p->killed = sig
            if p->state == SLEEPING:
                p->state = RUNNABLE
```

`tgkill()` 在设置挂起位之前先按 `tid` 在 `thread_queue` 中搜索线程，但写入的仍然是进程级 `sig_pending` 和 `killed`。这说明当前信号子系统以进程为中心，线程定向信号尚未抵达真正的递送阶段。

Trap 层对 `killed` 的消费发生在 `usertrap()`（`hsai/hsai_trap.c`）中：系统调用前若发现 `p->killed` 已置位，写入 `term_signal` 并直接调用 `exit()`。

2.6 退出、等待与资源回收

2.6.1 `exit` → `reparent` → `ZOMBIE` → `wait` → `freeproc` 回收链

资源回收链以 `exit()` 为起点：

1. 禁止 `initproc` 退出。
2. 通过 `get_file_ops()->close()` 关闭全部打开文件。
3. 获取全局 `parent_lock`，调用 `reparent(p)` 将子进程过继给 `initproc`。对于已是 `ZOMBIE` 的子进程直接调用 `freeproc()` 回收。
4. 唤醒父进程 `wakeup(p->parent)`。

5. 设置 `p->state = ZOMBIE`, `p->main_thread->state = t_ZOMBIE`。
6. 通过 `sched()` 永久离开运行态。

父进程通过 `wait()` 承担回收职责。`wait()` 扫描进程池查找属于当前进程的 ZOMBIE 子进程，取出退出状态后调用 `freeproc()` 真正回收；若未找到且未设置 `WNOHANG`，则通过 `sleep_on_chan(p, &p->lock)` 等待子进程退出。

2.6.2 `freeproc()` 的资源释放顺序

`freeproc()` 的释放顺序严格按照“进程对象 → 线程队列 → 地址空间 → 文件对象”展开：

Algorithm 2.3: `freeproc()` 的资源释放算法

```

1: 调用 pmem_free_pages(p->trapframe, 1) 释放进程活动 trapframe
2: for all 线程队列中的线程 t do
3:   if t 的 trapframe 与进程活动 trapframe 不同 then
4:     释放线程私有 trapframe
5:   end if
6:   if t 是附加线程 then
7:     解除附加线程内核栈映射并释放 kstack_pa
8:   end if
9:   设置 t->state = t_UNUSED，并将其放回 free_thread
10: end for
11: 调用 free_vma_list(p) 逐个释放 VMA
12: 调用 proc_freepagetable(p, p->sz) 释放用户页和多级页表
13: 对全部已打开文件逐描述符调用 get_file_ops()->close()
14: 重置进程槽位，例如 pid = 0、state = UNUSED、main_thread = NULL

```

关键细节：通过 `t->trapframe != p->trapframe` 判断避免重复释放主线程的 `trapframe`。附加线程在创建时分配了独立的 `trapframe` 和内核栈，回收时需单独释放。

2.6.3 `initproc` 对孤儿进程的回收

`wait()` 中，若当前进程是 `initproc`，则先调用 `reap_init_reparented_zombies(p)`。该函数扫描进程池，对所有 `parent == initproc` 且 `reparented_to_init` 标记为 1 的 ZOMBIE 子进程调用 `freeproc()`。这一机制确保所有被过继的僵尸进程最终都能得到回收。

2.7 当前设计的边界与局限

进程与线程管理仍有三方面改进空间：

- 线程创建与退出语义。`clone_thread()` 已具备独立现场构造框架，但 `exit(0)` 占

位和 `CLONE_CHILD_CLEAR_TID` 的 `TODO` 标记表明线程退出时的用户空间清零与通知逻辑尚未完成。

- **信号递送粒度。**当前实现能记录挂起信号并驱动终止或中断返回，但信号处理函数的用户态栈帧构造和线程定向信号递送尚未实现。
- **调度公平性。**当前 `scheduler()` 采用轮询扫描，没有维护优先级或时间片预算。同一进程内的多线程调度依赖线程在队列中的位置重排，而非独立的时间片管理。

当前代码建立了从创建、运行、阻塞、异步事件到退出回收的完整流程。通过 `proc_t`、`thread_t` 与 `cpu_t` 的分工，将任务模型、调度器、同步原语、信号和资源回收组织起来。

第三章 内存管理

3.1 总体架构

内存子系统分为三层：Buddy 管理物理页，页表层维护映射，VMA 层描述虚拟区域语义。

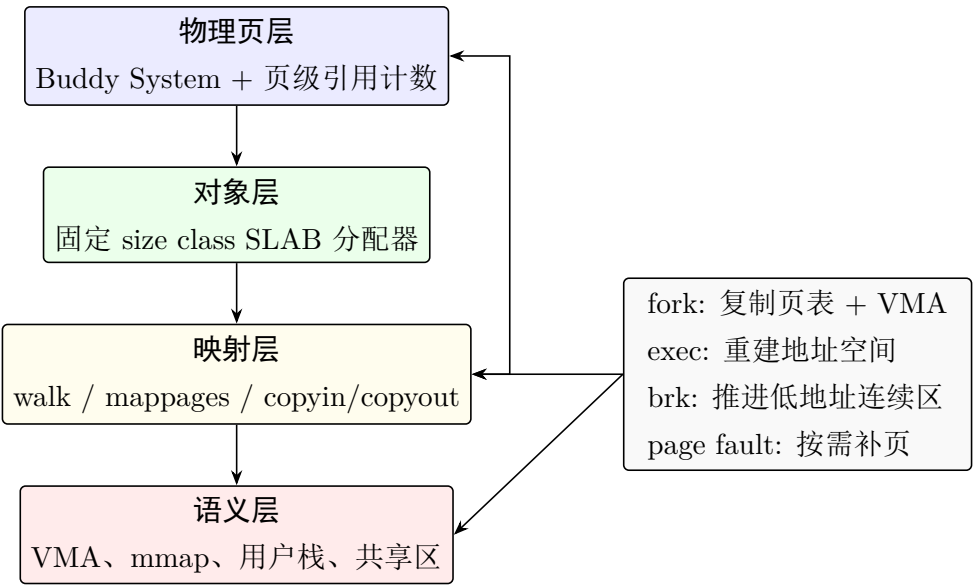


图 3.1: 内存子系统分层与主路径

初始化次序为 `pmem_init()` → `vmem_init()` → `slab_init()` (`kernel/sos_start_kernel.c`)。

3.2 Buddy System

3.2.1 数据结构

结构体	字段	类型	用途
buddy_system_t	—	—	全局 Buddy 系统实例，维护受管物理内存与各阶空闲链表。
buddy_system_t	mem_start/end	uint64	受管物理内存的起止地址。
buddy_system_t	total_pages	uint64	受管物理页总数。
buddy_system_t	bitmap	uint64*	页级位图，每位对应一页，其中 1 表示已使用，0 表示空闲。
buddy_system_t	nodes	buddy_node_t*	每页元数据数组，记录块起始地址、阶数和引用计数。

buddy_system_t	free_lists[11]	structlist	每个 order 对应的空闲块链表。
buddy_node_t	—	—	每页对应一项元数据，用于描述页所属块及空闲链表关系。
buddy_node_t	addr	uint64	所属块的起始物理地址。
buddy_node_t	order	int	块阶数，其中 -1 表示该页被元数据占用。
buddy_node_t	refcnt	int	引用计数，仅对 order 0 的物理页有效。
buddy_node_t	elem	list_elem	挂入对应空闲链表的链表节点。

表 3.2: Buddy System 核心数据结构

3.2.2 buddy_init() 自举过程

buddy_init() 将位图与节点数组元数据嵌入受管内存起始处，随后按最高 order 切分剩余内存并挂入空闲链表。

3.2.3 buddy_alloc(): 分配算法

```

buddy_alloc(order):
    for current_order = order to MAX_ORDER:
        if free_lists[current_order] 非空:
            取出一个空闲块节点 (地址 = addr, 阶数 = current_order)

            // 分裂: 将过大的块逐级对半分割
            while current_order > order:
                current_order--
                block_size = PGSIZE << current_order
                buddy_addr = addr + block_size
                将 buddy_addr 作为新块插入 free_lists[current_order]
                set_buddy_free(buddy_addr, current_order)

            // 清空分配的内存
            memset(addr, 0, PGSIZE << order)
            set_block_metadata(addr, order)
            set_block_refcnt(addr, order, 1)
            set_buddy_used(addr, order)
            return addr
    return NULL // 无可用内存

```

```

uint64 get_buddy_addr(uint64 addr, int order) {
    uint64 block_size = PGSIZE << order;

```

```

    return addr ^ block_size;
}

```

3.2.4 buddy_free(): 释放与合并算法

```

buddy_free(ptr, order):
    page_idx = (ptr - mem_start) / PGSIZE
    actual_addr = nodes[page_idx].addr // 从元数据恢复块起始
    actual_order = nodes[page_idx].order // 从元数据恢复块阶数

    // 重复释放检查
    if bitmap 中整个块已标记为空闲:
        return // 忽略重复释放

    set_buddy_free(actual_addr, actual_order)

    // 向上合并
    merge_addr = actual_addr
    merge_order = actual_order
    while merge_order < MAX_ORDER:
        buddy_addr = get_buddy_addr(merge_addr, merge_order)
        if is_buddy_available(buddy_addr, merge_order):
            从 free_lists[merge_order] 移除伙伴块
            merge_addr = min(merge_addr, buddy_addr)
            merge_order++
            set_block_metadata(merge_addr, merge_order)
        else:
            break

    将最终合并块插入 free_lists[merge_order]

```

3.2.5 PAGE_NUM 的双架构差异

架构	PAGE_NUM	总内存	定义位置
RISC-V	250 * 1024	≈ 1000 MiB	include/hal/riscv/riscv.h
LoongArch	100 * 1024	≈ 400 MiB	include/hal/loongarch/loongarch.h

表 3.4: 双架构的 PAGE_NUM 配置

其中，RISC-V 的受管物理内存约为 1024 MiB，LoongArch 约为 400 MiB。

3.2.6 页级引用计数

`pmem_inc_ref()` / `pmem_free_pages()` 通过 `refcnt` 管理共享页。引用计数仅对 `order 0` 有效，大于 1 时只递减。

3.3 SLAB 分配器

3.3.1 固定 size class 设计

SLAB 分配器固定档位如下：

```
uint64 slab_size_table[FIXED_CACHE_LEVEL_NUM] = {
    8, 16, 32, 64, 128, 256, 512, 1024
};
```

每个 `kmem_cache` 维护 `free_slab` 和 `full_slab` 两条链表。`slab` 占用一页，页头保存元数据。

档位 (bytes)	每 slab 对象数	利用率	典型用途
8	504	≈ 98%	极小结构体
16	252	≈ 98%	短指针数组
32	126	≈ 98%	链表节点
64	63	≈ 98%	共享内存元数据
128	31	≈ 97%	中等结构体
256	15	≈ 94%	文件系统元数据
512	7	≈ 87%	缓冲区
1024	3	≈ 75%	较大结构体

表 3.6: SLAB 固定档位的每页对象数

3.3.2 `slab_alloc()`: 分配流程

Algorithm 3.1: `slab_alloc()` 分配算法

- 1: 调用 `__slab_size(size)`，将请求大小对齐到最近固定档位
- 2: 通过 `__fine_kmem_cache` 查找对应的 `kmem_cache`
- 3: **if** 未找到对应 `cache` **then**
 - 4: `panic` ▷ 通常表示请求大小超过 1024
- 5: **end if**
- 6: **if** `cache->free_slab` 为空 **then**
 - 7: 调用 `__create_slab_with_size()` 创建新 `slab`，并挂入 `free_slab`

```

8: end if
9: 从 free_slab 中取出一个对象并更新空闲计数
10: if 该 slab 的空闲计数归零 then
11:     将该 slab 移入 full_slab
12: end if
13: 返回已分配对象

```

3.3.3 自举过程: simple_alloc → slab_init

simple_alloc() 是一个逐字节推进的 bump allocator:

```

simple_alloc(size):
    if simple_alloc_count + size > PAGE_SIZE:
        slab_boot_page = pmem_alloc_pages(1) // 新申请一页
        simple_alloc_count = 0
    alloc_ptr = slab_boot_page + simple_alloc_count
    memset(alloc_ptr, 0, size)
    simple_alloc_count += size
    return alloc_ptr

```

自举完成后设置 state = FULL, 后续分配均通过 slab_alloc()。kmalloc()/kfree() 仍为按页分配。

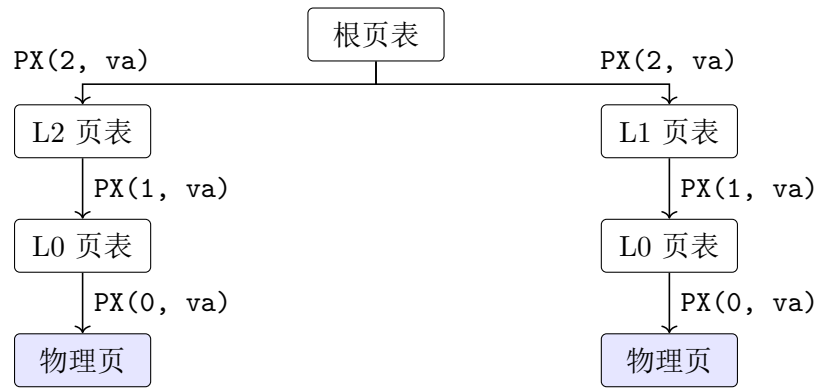
3.4 页表与地址映射

3.4.1 walk(): 多级页表遍历核心算法

```

walk(pt, va, alloc):
    assert(va < MAXVA)
    for level = PT_LEVEL-1 down to 1:
        pte = &pt[PX(level, va)] // 取当前级索引
        if *pte & PTE_V: // 有效, 找下一级
            pt = (pgtbl_t)(PTE2PA(*pte) | dmwin_win0)
        else if alloc: // 无效但允许分配
            pt = (pgtbl_t)pmem_alloc_pages(1)
            *pte = PA2PTE(pt) | PTE_WALK // 写入新页表物理地址
        else:
            return NULL
    pte = &pt[PX(0, va)] // 最后一级 (叶节点)
    return pte

```



RISC-V Sv39: 3 级 (9+9+9+12) **LoongArch:** 4 级 (9+9+9+9+12)

图 3.2: 多级页表遍历结构 (图中为 RISC-V 三级示例)

3.4.2 RISC-V 与 LoongArch 的页表差异

维度	RISC-V (Sv39)	LoongArch
PT_LEVEL	3	4
虚拟地址宽度	39 位 (9+9+9+12)	48 位 (9+9+9+9+12)
MAXVA	1L << 38	1ULL << 46
物理地址访问	直接使用物理地址	通过 DMW 直映窗口 (dmwin_win0)
权限位模型	PTE_R/W/X 显式设置	PTE_NR/NX 逆逻辑 + PTE_W/PLV3
内核映射需求	需显式映射 UART/VirtIO/- PLIC	DMW 窗口自动映射内核空间

表 3.8: 双架构页表差异对比

3.5 VMA 与用户地址空间

3.5.1 地址空间布局

用户地址空间自低到高划分为:

常量	值 (RISC-V)	定义位置
USER_MMAP_START	MAXUVA - 0x10000000 - 2*PGSIZE	include/hal/riscv/riscv.h
USER_STACK_TOP	MAXUVA - PGSIZE	同上
USER_STACK_SIZE	100 * PGSIZE	include/kernel/vma.h
TRAMPOLINE	MAXVA - PGSIZE	include/hal/riscv/riscv_memlayout.h

TRAPFRAME	TRAMPOLINE -	同上
	PGSIZE	

表 3.10: 用户地址空间关键常量

3.5.2 VMA 链表与区域语义

`struct vma` 保存区域类型、权限、起止地址、文件描述符与偏移。类型包括:

类型	用途	管理方式
NONE	哨兵头结点	永久存在于链表首位, 标记链表边界
MMAP	mmap 映射区 (文件映射/匿名映射)	<code>alloc_mmap_vma()</code> 创建, <code>munmap()</code> 裁剪
STACK	用户栈	<code>alloc_vma_stack()</code> 在 <code>exec()</code> 和 <code>init_process()</code> 中创建
SHARE	共享内存区域	通过共享内存系统调用创建

表 3.12: VMA 区域类型

`vma_init()` 创建双向循环链表, 头结点 NONE, 随后插入 MMAP 节点。

3.5.3 `alloc_vma` / `alloc_mmap_vma` 的地址选择策略

```
alloc_vma(p, type, addr, sz, perm, alloc, pa):
    start = PGROUNDUP(addr)
    end = PGROUNDUP(addr + sz)
    // 遍历 VMA 链表, 寻找插入点
    find_vma = p->vma->next
    while find_vma != p->vma:
        if end <= find_vma->addr: // 新区域完全在 find_vma 之前
            break
        else if start >= find_vma->end: // 新区域在 find_vma 之后, 继续
            find_vma = find_vma->next
        else:
            panic("vma address overflow")
    // 创建并插入新 VMA
    vma = pmem_alloc_pages(1)
    插入到 find_vma 之前
    return vma
```

`alloc_mmap_vma()` 在 `start == 0` 时从首个 MMAP VMA 向下布置。

3.5.4 mmap / munmap 的实现流程

`munmap()` 处理完整覆盖、前缀/后缀、中间段三种重叠。

Algorithm 3.2: `munmap()` 的 VMA 分裂算法

- 1: 在 VMA 链表中找到与区间 $[start, end)$ 重叠的映射项
- 2: **if** 解除范围完整覆盖原 VMA **then**
- 3: 删除该 VMA，并解除整段映射
- 4: **else if** 解除范围与原 VMA 前缀重叠 **then**
- 5: 将原 VMA 起点推进到 end
- 6: 同步调整文件偏移 `f_off`
- 7: **else if** 解除范围与原 VMA 后缀重叠 **then**
- 8: 将原 VMA 终点收缩到 $start$
- 9: **else**
- 10: 保留前段区间 $[addr, start)$
- 11: 新建后段区间 $[end, old_end)$
- 12: 设置后段的 `f_off += (end - start)`
- 13: **end if**
- 14: 对被解除的子区间执行页取消映射与物理页回收

3.5.5 fork() 时 VMA 的共享与私有处理

```
if vma_is_shared_mapping(vma): // MAP_SHARED 类型
    pmem_inc_ref(pa) // 增加页级引用计数
    mappages(new, start, pa, ...) // 同一物理页映射到子进程
else: // 私有映射
    mem = pmem_alloc_pages(1) // 分配新物理页
    memmove(mem, pa, PGSIZE) // 复制内容（写时复制尚未实现）
    mappages(new, start, mem, ...)
```

3.5.6 pagefault_handler 的决策流程

```
pagefault_handler(addr):
    p = myproc()
    if addr >= MAXVA: return -1

    if addr <= p->sz:
        // 低地址连续区内缺页
        flag = 1
        perm = PTE_R | PTE_W | PTE_X | PTE_D | PTE_U
    else:
```

```
// 遍历 VMA
find_vma = find_mmap_vma(p->vma)
while find_vma != p->vma:
    if addr within find_vma:
        flag = 1
        perm = find_vma->perm | PTE_U
        break
    find_vma = find_vma->next

if !flag: return -1

pa = pmem_alloc_pages(1) // 分配物理页
memset(pa, 0, PGSIZE) // 清零新页
perm = PTE_R | PTE_W | PTE_X | PTE_D | PTE_U // 权限统一放宽
if mappages(p->pagetable, aligned_addr, pa, PGSIZE, perm) < 0:
    return -1
return 0
```

当前缺页处理在补页时统一放宽权限。

3.6 小结

本章围绕内存管理主线，梳理了 Buddy、SLAB、页表与 VMA 四层机制。Buddy 提供页级物理内存分配与回收，SLAB 负责固定档位小对象管理；页表层承担虚实地址映射，VMA 层补充区域权限、用途与文件关联语义。

几层机制协同后，系统形成了从物理页管理、内核对象分配到用户地址空间组织的完整内存管理框架，也为 `mmap()`、`munmap()`、`fork()` 与缺页处理等路径提供了统一支撑。

第四章 硬件抽象层与异常处理机制

4.1 分层架构与代码边界

硬件抽象层(HAL)与硬件无关内核接口(HSAI)的分工,是本系统在双架构(RISC-V 64 与 LoongArch 64)条件下保持内核主干代码统一性的关键设计决策。

HAL 层承担以下三类底层事务:

1. 启动入口与特权级准备: `hal/riscv/entry.S`、`hal/riscv/start.c`、`hal/loongarch/entry.S`, 完成栈准备、CSR 配置后转入统一的 C 入口 `sos_start_kernel()`。
2. Trap 汇编桥接: 两套 `trampoline.S` (用户态陷入/返回)、`kernelvec.S` (内核态陷入/返回) 以及 LoongArch 的 `tlbrefill.S` (TLB 重填)。
3. 上下文切换原语: `swtch()`, 分别在 `hal/riscv/switch.S` 和 `hal/loongarch/swtch.S` 中实现, 仅保存 `ra`、`sp` 与被调用者保存寄存器。

HSAI 层将这些差异统一为少量接口函数, 主要位于 `hsai/hsai_trap.c` 与 `hsai/hsai_mem.c`。其设计原则是: 内核核心层(调度、系统调用、缺页、定时器)只调用 HSAI 函数, 不直接读写架构 CSR。例如 `hsai_get_arg()` 统一提取 `a0-a7`, `hsai_usertrapret()` 统一准备返回用户态, `hsai_config_pagetable()` 统一写入页表根寄存器。

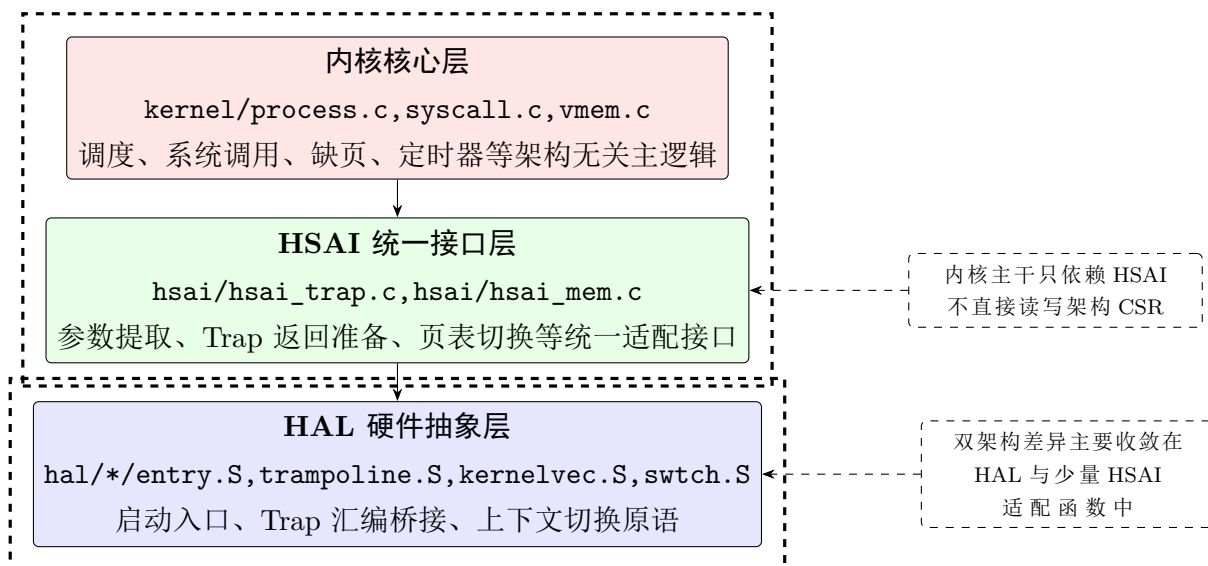


图 4.1: HAL → HSAI → Kernel 三层关系

4.2 双架构启动链

4.2.1 RISC-V 启动链

RISC-V 的启动入口在 `_entry` (`hal/riscv/entry.S`), 将 `sp` 指向 `entry_stack` 后调用 `start()`。

`start()` (`hal/riscv/start.c`) 在 M 态下设置 `mstatus.MPP` 为 S 态、配置 `mepc` 指向 `sos_start_kernel()`、清空 `satp`、通过 `medeleg` 和 `mideleg` 委托异常和中断给 S 态、配置 PMP，最后以 `mret` 跳转到 `sos_start_kernel()`。SBI 路径直接调用 `sos_start_kernel()`，由 QEMU 内置 OpenSBI 代为完成 M 态 CSR 配置。

4.2.2 LoongArch 启动链

LoongArch 入口在 `hal/loongarch/entry.S` 的 `_entry`，进入 C 代码前在汇编层完成硬件配置：

1. 配置 `DMWIN0` (CA, PLV0, 地址窗口 0x9000 xxxx xxxx xxxx) 和 `DMWIN1` (UA, PLV0, 地址窗口 0x8000 xxxx xxxx xxxx)，建立直接映射窗口。
2. 配置 `CRMD` (PLV=0, IE=0, PG=1)，开启分页模式。
3. 清空 `TLBREENTRY` 并执行 `invtlb` 使所有 TLB 项失效。
4. 读取 `CPUID` 构造每个 CPU 的启动栈。
5. 仅引导 CPU (`CPUID==0`) 清零 `.bss` 段。
6. 跳转到 `sos_start_kernel()`。

表 4.1: 双架构启动链对比

阶段	RISC-V (非 SBI)	LoongArch
入口文件	<code>hal/riscv/entry.S</code>	<code>hal/loongarch/entry.S</code>
M 态准备	<code>mstatus/mepc/medeleg/mideleg/PMP</code>	无 M 态，直启 S 态
地址映射	<code>satp=0</code> 关闭分页	<code>DMWIN0/1</code> 建立直接映射窗口
分页开启	在 <code>sos_start_kernel()</code> 内	入口配置 <code>CRMD.PG=1</code>
BSS 清零	在链接脚本中	入口汇编自行清零
特权级转换	<code>mret</code> (M $\rightarrow$ S)	无需转换 (直启 S 态)
进入内核	<code>sos_start_kernel()</code> 作为函数调用	同左

4.3 Trapframe 与 Context 的分工

`include/hsai/trap.h` 定义了 `struct trapframe`，用于保存完整的用户态执行现场。`include/kernel/context.h` 定义了 `context_t`，仅保存调度切换所需的最小内核现场。

表 4.3: trapframe 域布局对比

域	RISC-V 偏移	LoongArch 偏移
<code>kernel_satp</code>	0	280
<code>kernel_sp</code>	8	248

kernel_trap	16	256
epc/era	24	264
kernel_hartid	32	272
ra	40	0
sp	48	16
a0	112	24
a1-a7	120-168	32-80
s0-s11	96-248	176-240
t0-t6	72-280	88-152

两类上下文分工如下：

- **trapframe**: 由 `uservec/userret` 在 `trampoline` 页中读写，保存用户态全部寄存器。进程从用户态陷入内核时，硬件跳转到 `uservec`，由汇编将现场保存至此。
- **context_t**: 由 `swtch()` 在 `hsai_swtch()` 中读写，仅保存 `ra`、`sp`、`s0-s11` (RISC-V) 或 `fp` (LoongArch)，不包含页表信息或用户态寄存器。

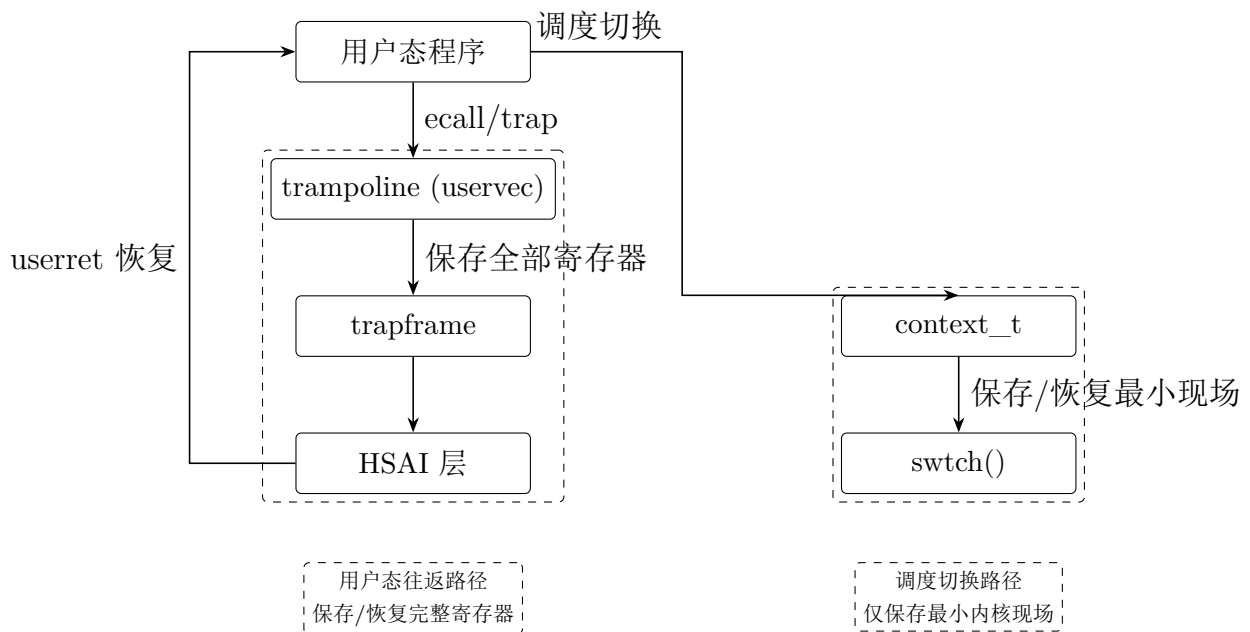


图 4.2: trapframe 与 context_t 的分工

4.4 用户态 Trap 主路径

用户态 Trap 主路径涵盖系统调用、缺页异常和设备中断三类事件。整体流程如算法 4.4 所示。

Algorithm 4.1: 用户态 Trap 主路径

```

1: 用户态执行过程中发生 ecall、异常或设备中断
2: 进入 uservec()，保存通用寄存器并切换到内核页表
3: 调用 usertrap()，保存异常 PC (sepc 或 era)
4: 调用 devintr() 检查是否为设备或时钟中断
5: if devintr() 返回非 0 then
6:   if 中断类型为时钟中断 then
7:     更新用户态时间统计
8:     调用 yield() 让出 CPU
9:   end if
10: 调用 hsai_usertrapret() 准备返回用户态
11: else
12:   if 异常原因是系统调用 then
13:     将 trapframe->epc/era 前移到下一条指令
14:     开中断并调用 syscall() 分发系统调用
15:   else if 异常原因是缺页异常 then
16:     调用 pagefault_handler() 尝试补页
17:     if 补页失败 then
18:       将进程标记为 SIGSEGV
19:     end if
20:   else
21:     打印异常信息并将进程标记为终止
22:   end if
23: 调用 hsai_usertrapret()，写回返回所需寄存器与状态
24: end if
25: 跳转到 userret()，执行 sret/ertn 返回用户态

```

4.4.1 usertrap() 异常分类

`usertrap()` (`hsai/hsai_trap.c`) 首先将内核 Trap 向量切换到 `kernelvec` (防止处理过程中再次陷入)，随后保存异常 PC (`sepc` 或 `era`)，然后根据异常原因分类。

表 4.5: RISC-V scause 与 LoongArch estat.ecode 对比

事件	RISC-V scause	LoongArch estat.ecode
系统调用	UserEnvCall (8)	0xb
Load 页故障	LoadPageFault (13)	0x1
Store 页故障	StorePageFault (15)	0x2

指令未对齐	InstructionMisaligned (0)	0x9
非法指令	IllegalInstruction (2)	0xa
断点	Breakpoint (3)	0xc
时钟中断	0x8000000000000005	TI_VEC 位
外部中断	(scause & 0xff) == 9	HWI_VEC 位

分类逻辑的伪代码如下（RISC-V 分支）：

```

usertrap(p->trapframe):
    stvec = kernelvec // 内核态陷入入口
    trapframe->epc = sepc // 保存异常PC

    which_dev = devintr()
    if which_dev != 0: // 设备/时钟中断
        if which_dev == 2: // 时钟中断
            p->utime++
            yield()
            hsai_usertrapret()
            // 其他中断后续处理

    else: // 异常
        if cause == UserEnvCall: // 系统调用
            trapframe->epc += 4 // 跳过ecall
            intr_on()
            syscall(trapframe)
        elif cause == LoadPageFault
            or cause == StorePageFault:
            pagefault_handler(stval)
        else: // 不可恢复
            print_diagnostics()
            p->term_signal = SIGSEGV/SIGILL/SIGBUS
            exit(0)

```

LoongArch 分支结构相同，差异在于：异常 PC 字段为 era 而非 epc；系统调用判定使用 (estat&CSR_ESTAT_ECODE)>>16==0xb；页故障地址取自 badv 而非 stval；断点异常（ecode 0xc）先于系统调用判断，直接 era+=4 后跳过。

4.4.2 devintr() 中断分发

devintr()（hsai/hsai_trap.c）将中断分为外部中断和时钟中断两类。

RISC-V 路径：检查 scause 高位置 1 且低 8 位为 9（S 态外部中断）则调用 plic_claim() 获取 IRQ 并路由至对应驱动；检查 scause==0x8000000000000005（S 态时钟

中断) 则调用 `timer_tick()`。

LoongArch 路径: 读取 `estat` 和 `ecfg`, 执行 `estat&ecfg` 位与得到有效中断向量。若匹配 `HWI_VEC` (硬件中断) 则打印中断信息 (当前实现为初步, 尚未接入完整设备中断处理); 若匹配 `TI_VEC` (定时器中断) 则调用 `timer_tick()`, 写入 `TICLR` 清除中断标志。

4.4.3 缺页处理

`pagefault_handler()` (`hsai/hsai_trap.c`) 判断故障地址是否在进程堆范围或某个 VMA 区间内: 若在堆内则分配单页物理内存并以 `PTE_R|PTE_W|PTE_X|PTE_U` 权限建立映射; 若在 VMA 内则按 `find_vma->perm|PTE_U` 建立映射; 分配失败或地址不在任何合法区域时返回 -1, 由 `usertrap()` 将进程标记为 `SIGSEGV`。

4.5 内核态 Trap

`kerneltrap()` (`hsai/hsai_trap.c`) 处理内核态异常。与 `usertrap()` 的关键差异在于:

- 进入时检查当前模式是否为 S 态 (RISC-V 检查 `sstatus.SPP`, LoongArch 检查 `prmd.PPLV`), 否则 `panic`。
- 进入时要求中断已关闭 (`intr_get()==0`)。
- 不处理系统调用和用户态缺页, 仅处理设备中断和时钟中断。
- 未识别的异常打印调试信息后 `panic`。
- 时钟中断递增 `p->ktime`, 若进程状态为 `RUNNING` 则调用 `yield()`。
- 返回前恢复 `sepc/sstatus` (或 `era/prmd`), 因为 `yield()` 可能导致上下文切换。

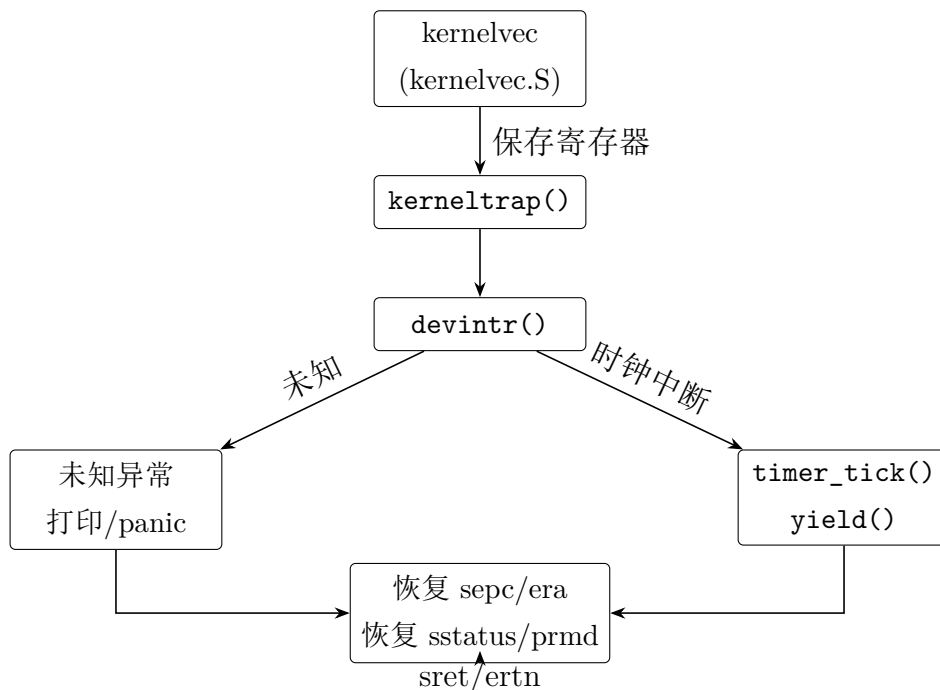


图 4.3: `kerneltrap()` 处理流程

4.6 LoongArch TLB Refill

LoongArch 缺乏 RISC-V 的硬件页表遍历能力, TLB 未命中时由软件在 `hal/loongarch/tlbrefill.S` 的 `handle_tlbr()` 中处理。该函数运行在专门的 TLB Refill Exception 入口 (`tlbrentry`), 不使用常规的 `eentry`。

```
handle_tlbr():
    // 保存 t0 到 TLBRSAVE
    csrwr t0, TLBRSAVE

    // 读取 PGD (根页表地址)
    csrrd t0, PGD

    // 四级页表遍历
    lddir t0, t0, 3 // 读 PGD[VPN3] -> PMD
    srli.d t0, t0, 12
    slli.d t0, t0, 12
    lddir t0, t0, 2 // 读 PMD[VPN2] -> PUD
    srli.d t0, t0, 12
    slli.d t0, t0, 12
    lddir t0, t0, 1 // 读 PUD[VPN1] -> PTE 基址
    srli.d t0, t0, 12
    slli.d t0, t0, 12

    // 加载页表项
    ldpte t0, 0 // 加载偶数页 PTE
    ldpte t0, 1 // 加载奇数页 PTE

    // 填充 TLB
    tlbfill

    // 恢复 t0 并返回
    csrrd t0, TLBRSAVE
    ertn
```

`lddir` 指令从页表目录中加载下一级页表基址, `ldpte` 加载页表项到 TLB 缓冲区, `tlbfill` 写入 TLB。与 RISC-V 硬件页表遍历的对比如下:

表 4.7: TLB 未命中处理: RISC-V vs LoongArch

特性	RISC-V	LoongArch
----	--------	-----------

页表遍历实现	硬件（页表遍历器）	软件（ <code>handle_tlbr</code> ）
TLB 缺失入口	无专用入口	<code>tlbrentry</code>
处理路径长度	无额外软件开销	约 $9 + 3N$ 条指令（ N = 页表级数）
页表级数	Sv39 (3 级)	4 级（使用 <code>lddir</code> 指令）
影响范围	无特殊设计需求	要求所有用户页表在内存中可达

这一差异要求 LoongArch 在 `hsai_config_pagetable()` 中额外配置 `pwcl` 和 `pwch` 寄存器，描述页表各级目录的宽度和基址。

4.7 hsai_usertrapret() 返回路径

`hsai_usertrapret()` (`hsai/hsai_trap.c`) 负责从内核态准备返回用户态，跳转到 `trampoline` 中的 `userret`。

Algorithm 4.2: hsai_usertrapret() 返回准备流程

- 1: 进入 `hsai_usertrapret()`
- 2: 关闭中断，避免返回准备过程中再次陷入
- 3: 重置用户态 Trap 入口，使后续用户态异常重新进入 `uservec()`
- 4: 将当前内核栈地址写回 `trapframe->kernel_sp`
- 5: 将当前内核页表根写回 `trapframe` 中对应字段
- 6: 将内核 Trap 处理入口写回 `trapframe->kernel_trap`
- 7: 设置返回用户态所需的 `sstatus/prmd`
- 8: 设置异常返回地址 `sepc/era`
- 9: 跳转到 `trampoline` 中的 `userret()`

准备完成后，通过函数指针跳转到 `trampoline` 内的 `userret`。参数传递和特权级返回指令的架构差异如下：

表 4.9: RISC-V `sret` vs LoongArch `ertn`

项目	RISC-V	LoongArch
返回地址寄存器	<code>sepc</code>	<code>era</code>
页表寄存器	<code>satp</code> （写入 <code>a1</code> ）	<code>pgdl</code> （写入 <code>a1</code> ）
Trapframe 寄存器	<code>a0 = TRAPFRAME</code>	<code>a0 = TRAPFRAME</code>
返回指令	<code>sret</code>	<code>ertn</code>
状态恢复	<code>sstatus</code> （由 <code>sret</code> 自动恢复）	<code>prmd</code> （由 <code>ertn</code> 自动恢复）

Trampoline 段	hal/riscv/trampoline.S	hal/loongarch/trampoline.S
--------------	------------------------	----------------------------

`userret` 在汇编层完成用户页表切换、用户通用寄存器恢复，执行 `sret` 或 `ertrn` 恢复 CPU 状态并跳转到 `sepc/era` 处继续执行。

第五章 系统调用与用户态接口

5.1 设计目标

系统调用层是用户态与内核之间的 ABI 边界。系统调用设计遵循以下两条原则：

1. 以 Linux 风格系统调用号为起点。include/kernel/syscall_ids.h 中的编号直接对应 Linux 的 syscall table，按 BusyBox、glibc、Iozone、LTP、pthread 等工作负载逐步补齐。内核无需对用户程序进行源码级修改，即可运行预编译的 Linux ELF 二进制。
2. 双架构差异隔离在陷入指令与异常码层面。参数提取、编号分发、错误返回和兼容路径处理集中在 hsai/hsai_trap.c、kernel/syscall.c 与 kernel/vmem.c 中；架构相关的差异只出现在 user/*/usys.S 的调用桩中。

系统调用与用户态接口按以下分层组织：

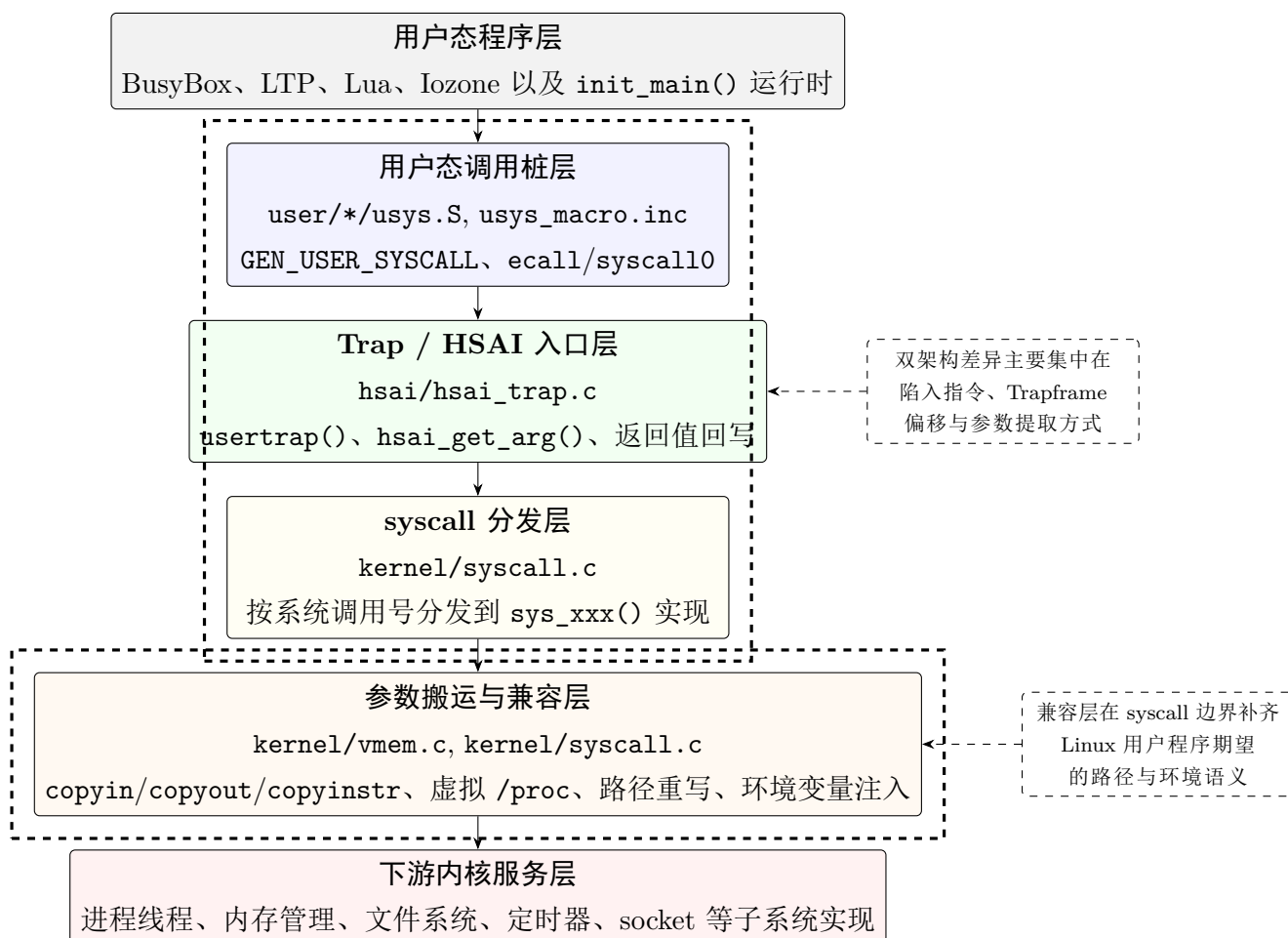


图 5.1: 系统调用与用户态接口的分层结构

5.2 ABI 入口与用户态调用桩

5.2.1 双架构调用约定

用户态调用桩由宏 `GEN_USER_SYSCALL` 生成。RISC-V 版本位于 `user/riscv/include/usys_macro.inc`，LoongArch 版本位于 `user/loongarch/include/usys_macro.inc`。两者遵循相同的抽象语义：将系统调用号写入约定寄存器，执行陷入指令，返回用户态调用者。

表 5.1: 当前系统调用 ABI 的双架构约定

含义	RISC-V 64	LoongArch 64
系统调用号	a7	\protect\TU\textdollar a7
参数寄存器	a0–a5	\protect\TU\textdollar a0– \protect\TU\textdollar a5
返回值寄存器	a0	\protect\TU\textdollar a0
陷入指令	ecall	syscall0
返回指令	ret	jirl\protect\TU\textdollar ze ro,\protect\TU\textdollar ra, 0

`user/riscv/usys.S` 与 `user/loongarch/usys.S` 覆盖内核自带用户程序所需的最小接口集合，例如 `sys_execve`、`sys_openat`、`sys_read`、`sys_mmap`、`sys_getdents64` 等。

5.2.2 GEN_USER_SYSCALL 宏

调用桩生成宏（RISC-V 版本）如下：

```
.macro GEN_USER_SYSCALL name, num
.globl \name
\name:
    li a7, \num // 将系统调用号装入 a7
    ecall // 陷入内核
    ret // 返回用户态调用者
.endm
```

LoongArch 版本仅将 `ecall` 替换为 `syscall0`，确保所有用户态系统调用使用统一的陷入点，且不修改 `a0–a5` 参数寄存器的值。

5.3 从 Trap 到 syscall() 的统一分发

5.3.1 syscall() 分发器

syscall() (kernel/syscall.c) 调用 hsai_get_arg() 提取 a[0..7], 以 a[7] 为编号经 switch 分发到 sys_xxx(), 返回值写入 trapframe->a0。

```
void syscall(struct trapframe *trapframe) {
    for (int i = 0; i < 8; i++)
        a[i] = hsai_get_arg(trapframe, i);

    long long ret = -1;
    switch (a[7]) {
    case SYS_write: ret = sys_write(a[0], a[1], a[2]); break;
    case SYS_read: ret = sys_read(a[0], a[1], a[2]); break;
    case SYS_openat: ret = sys_openat(a[0], (char*)a[1], ...); break;
    case SYS_mmap: ret = sys_mmap(a[0], a[1], a[2], ...); break;
    // ... (约 100+ 个 case)
    default: ret = -ENOSYS; break;
    }
    trapframe->a0 = ret;
}
```

hsai_get_arg() 统一参数抽象的实现如下:

```
uint64 hsai_get_arg(struct trapframe *trapframe, uint64 num) {
    switch (num) {
    case 0: return trapframe->a0;
    case 1: return trapframe->a1;
    // ... a2..a6
    case 7: return trapframe->a7;
    }
}
```

RISC-V 与 LoongArch 的 trapframe 中 a0-a7 偏移不同, HSAI 函数屏蔽了这一差异。

5.4 参数搬运机制

指针参数需要借助 kernel/vmem.c 提供的用户空间数据搬运函数。

表 5.3: 参数搬运函数职责分工

函数	功能	典型用途
----	----	------

copyin()	从用户空间复制任意长度数据到内核	sys_writev 复制 iovec
copyout()	从内核复制数据到用户空间	sys_gettimeofday、sys_statx
copyinstr()	从用户空间复制以 \0 结尾的字符串	sys_openat 复制路径名
fetchaddr()	从用户地址读取一个指针宽度整数	sys_execve 读取 argv[i]
fetchstr()	从用户空间读取字符串（基于 copyinstr）	sys_execve 读取参数值

跨页搬运基于 walkaddr()（kernel/vmem.c）实现：将目标用户地址向下页对齐后逐页翻译得到物理地址，再逐页执行 memmove()。搬运函数不感知架构差异，walkaddr() 内部已处理了双架构页表翻译路径的差异。

5.5 系统调用分类

当前系统调用按编号分布在 include/kernel/syscall_ids.h 中，实现集中于 kernel/syscall.c。从主要工作负载来看，它们按以下子系统组织：

表 5.5: 系统调用的主要类别

类别	代表接口	主要依赖
进程与执行	sys_fork、sys_clone、sys_wait、sys_execve、sys_exit、sys_kill、sys_tgkill、sys_sched_yield	进程/线程管理、信号、ELF 装载器
内存与时间	sys_brk、sys_mmap、sys_munmap、sys_mprotect、sys_clock_gettime、sys_clock_nanosleep、sys_gettimeofday、sys_settimer	页表、VMA、定时器、调度
文件与路径	sys_openat、sys_read、sys_write、sys_readv、sys_pread、sys_getdents64、sys_fstat、sys_statx、sys_readlinkat、sys_mkdirat、sys_unlinkat、sys_mount、sys_ount、sys_renameat2	VFS/EXT4、路径解析、文件对象层

信号与同步	<code>sys_rt_sigaction</code> 、 <code>sys_rt_sigpr</code> 信号处理、 <code>futex</code> 、等待队列 <code>ocmask</code> 、 <code>sys_futex</code> 、 <code>sys_ppoll</code> 、 <code>sys_pselect6_time32</code> 、 <code>sys_even</code> <code>tfd2</code> 、 <code>sys_epoll_create1</code> 、 <code>sys_</code> <code>signalfd4</code> 、 <code>sys_timerfd_create</code> 、 <code>sys_pidfd_open</code>
网络与系统信息	<code>sys_socket</code> 、 <code>sys_bind</code> 、 <code>sys_connec</code> socket 层、系统信息填充 <code>t</code> 、 <code>sys_accept</code> 、 <code>sys_sendto</code> 、 <code>sys_re</code> <code>cvfrom</code> 、 <code>sys_sysinfo</code> 、 <code>sys_uname</code> 、 <code>sys_getrandom</code> 、 <code>sys_prlimit64</code>

5.6 兼容层设计

兼容层使预编译的 Linux ELF 程序无需修改即可运行，在系统调用边界上补齐路径、元数据、环境变量和挂载语义。

5.6.1 BusyBox 虚拟文件系统

BusyBox 的许多命令依赖读取 `/proc` 和 `/etc` 下的虚拟文件。内核通过 `is_busybox_virtual_path()` (`kernel/syscall.c`) 拦截这些路径，在 `sys_openat()`、`sys_fstatat()`、`sys_statx()` 等函数中直接返回合成数据，不经过底层 EXT4 文件系统。

当前支持的虚拟路径包括：

```
/proc /proc/self /proc/mounts
/proc/meminfo /proc/cpuinfo /proc/config.gz
/proc/self/maps /proc/self/exe /proc/self/status
/proc/self/stat /proc/[pid]/stat /proc/sys/kernel/pid_max
/etc/passwd /etc/group /dev/misc/rtc
/proc/sys/fs/pipe-user-pages-soft /proc/sys/kernel
```

处理流程如算法 5.6.1 所示。

Algorithm 5.1: BusyBox 虚拟文件处理流程

- 1: 进入 `sys_openat()` 或相关路径查询接口
- 2: 读取并规范化目标路径
- 3: **if** `is_busybox_virtual_path()` 判定该路径为虚拟路径 **then**
- 4: 构造合成文件对象或合成元数据
- 5: 将文件类型标记为 `FD_BUSYBOX`
- 6: 直接返回虚拟文件描述符或查询结果
- 7: **else**
- 8: 将请求下发到底层文件系统

```

9:    调用 vfs_ext4_openat() 或对应 EXT4 路径处理函数
10:   返回真实文件描述符或查询结果
11: end if

```

以 `fill_busybox_virtual_kstat()` (`kernel/syscall.c`) 为例,它在 `sys_fstatat()` 中填充 `kstat` 结构的 `st_mode`、`st_size` 等字段。例如 `/proc` 构造为权限 `0040777` 的目录, `/proc/meminfo` 构造为权限 `0100644` 的普通文件。

5.6.2 LD_LIBRARY_PATH 注入

`sys_execve()` (`kernel/syscall.c`) 在复制 `argv` 与 `envp` 时,预先在环境变量头部注入 `LD_LIBRARY_PATH=/glibc/lib:/musl/lib:`,并跳过用户态已提供的同名环境变量。

```

sys_execve(path, uargv, uenvp):
    // 复制参数数组到 argv[]
    for i = 0..MAXARG:
        fetchaddr(uargv + i*8, &uarg)
        if uarg == 0: break
        fetchstr(uarg, argv[i], PGSIZE)

    // 注入 LD_LIBRARY_PATH
    envp[0] = "LD_LIBRARY_PATH=/glibc/lib:/musl/lib:"
    env_count = 1

    // 复制用户环境变量 (跳过同名项)
    for i = 0..:
        fetchaddr(uenvp + i*8, &uenv)
        if uenv == 0: break
        if strncmp(env_str, "LD_LIBRARY_PATH=", 16) == 0:
            continue // 跳过
        envp[env_count++] = env_str

    ret = exec(path, argv, envp)
    // 清理分配的内存

```

这一机制使后续装载器可以在相同 ABI 边界下同时服务 `glibc` 与 `musl` 风格程序,而不要求每个用户程序在启动前都手工准备共享库搜索路径。

5.6.3 /dev/shm 重映射

共享内存工作负载 (如 LTP 的 `shmctl` 测例) 期望 `/dev/shm` 存在于 `tmpfs` 上。`get_absolute_path()` (`kernel/fs/inode.c`) 在路径规范化时将 `/dev/shm` 及其子路径重写

到 /ltp-shm，统一重定向到根文件系统下。

5.6.4 Synthetic Mount 机制

`sys_mount()` 和 `sys_umount()` (`kernel/syscall.c`) 支持 `tmpfs`、`ext4` 和 `vfat` 三种类型。对于非根目录的挂载请求，内核不注册第二个文件系统实例，而是通过 `set_synthetic_ro_mount()` 维护 `synthetic mount` 状态。

```
sys_mount(special, dir, fstype, flags, data):
    copyinstr 路径到内核缓冲区
    get_absolute_path(dir) -> abs_path

    if fstype == "tmpfs":
        set_synthetic_ro_mount(abs_path)
        synthetic_mount_readonly = (flags & MS_RDONLY)
        return 0

    if fstype in ("ext4", "vfat") and abs_path != "/":
        set_synthetic_ro_mount(abs_path) // 非根目录：模拟挂载
        synthetic_mount_readonly = (flags & MS_RDONLY)
        return 0

    // 根目录：真实挂载
    return fs_mount(TMPDEV, fstype, abs_path, flags, data)
```

后续对该路径的写入请求会被 `reject_path_write_on_ro_mount()` 拦截，并返回 `-EROFS`。`sys_umount()` 检测到 `synthetic mount` 路径时，直接清除相应挂载状态，而不操作底层 VFS。

5.7 本章小结

当前系统调用层形成了完整的调用路径：用户态调用桩触发陷入，Trap 层完成异常识别，`syscall()` 分发到 `sys_xxx()`，借助 `copyin/copyout/copyinstr` 搬运数据，最后恢复到用户态。兼容层在 `syscall` 边界提供虚拟 `/proc`、路径重写、环境变量注入和 `synthetic mount` 等机制，使系统调用层成为控制流入口与用户态可运行性的关键承载点。

第六章 文件系统与程序装载

6.1 总体架构

文件系统与程序装载的实现分布在 `kernel/fs/` 与 `kernel/exec.c` 中，按分层方式组织：

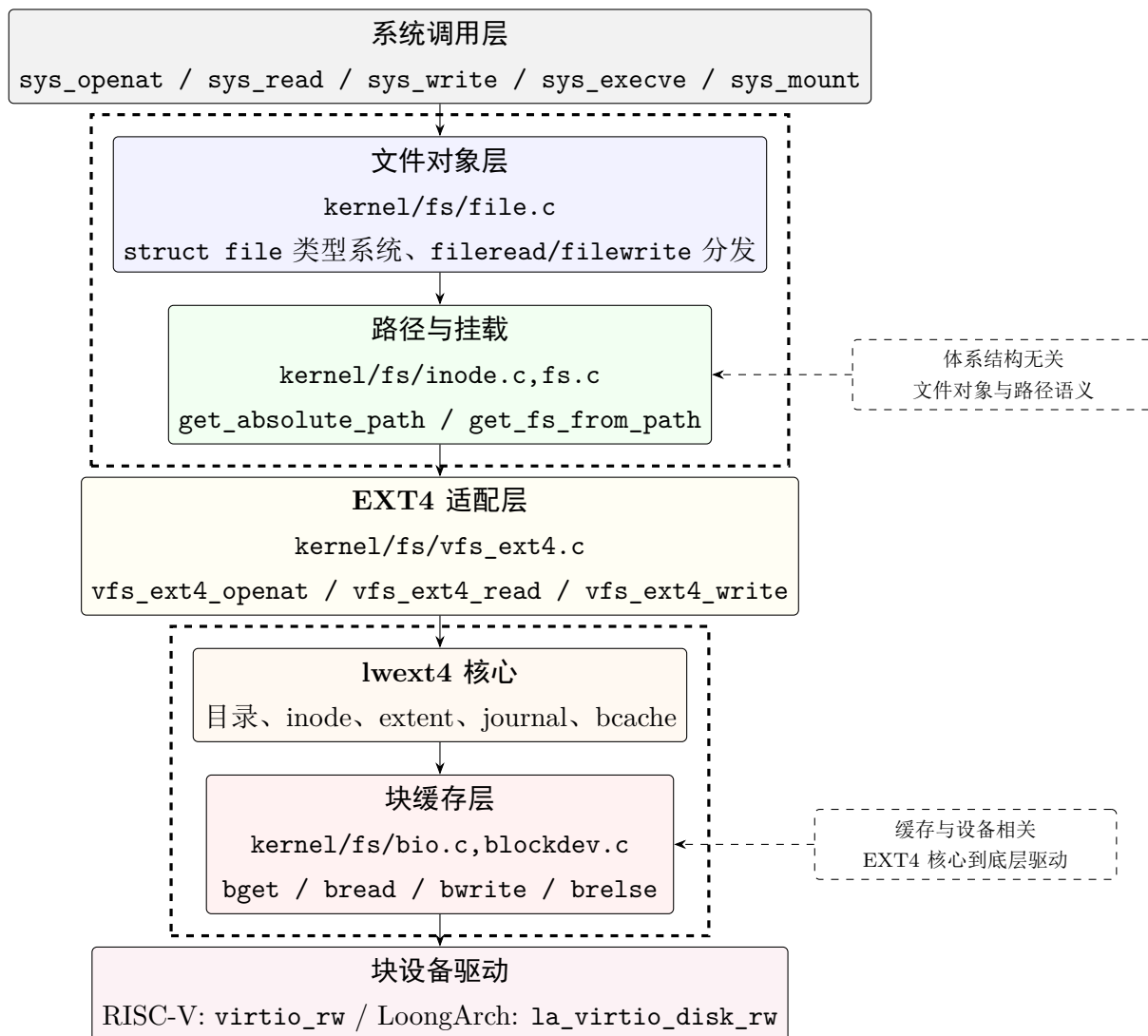


图 6.1: 文件系统访问栈的分层结构

各层职责如下：

- **文件对象层** (`kernel/fs/file.c`)：提供统一的 `struct file` 抽象，管理文件描述符的分配、复制、关闭和类型分发 (`FD_PIPE / FD_REG / FD_DEVICE / FD_BUSYBOX / FD_SOCKET` 等)。
- **路径与挂载** (`kernel/fs/inode.c, fs.c`)：将用户传入的路径归一化为绝对路径，通过最长前缀匹配选择挂载点。

- **EXT4 适配层** (`kernel/fs/vfs_ext4.c`): 将文件对象层的语义请求转换为 `lwext4` 库调用。
- **lwext4 核心**: 完成 EXT4 文件系统格式解析、inode 操作、目录遍历和日志 (journal) 管理。
- **块缓存层** (`kernel/fs/bio.c`): 以 LRU 链表维护磁盘块的缓存副本, 对上层提供 `bread / bwrite` 接口, 屏蔽底层设备差异。
- **块设备驱动**: RISC-V 通过 MMIO 驱动 VirtIO 设备, LoongArch 通过 PCI/virtio-pci 方式驱动。

6.2 文件对象层

6.2.1 struct file 的类型系统

`include/kernel/fs/file.h` 中的 `struct file` 是文件访问的统一抽象。其 `f_type` 字段标识对象类型:

类型常量	数值	用途
<code>FD_NONE</code>	0	未分配
<code>FD_PIPE</code>	1	管道
<code>FD_REG</code>	2	普通文件
<code>FD_DEVICE</code>	3	字符设备
<code>FD_BUSYBOX</code>	4	BusyBox 虚拟文件
<code>FD_SOCKET</code>	5	Socket (stub)
<code>FD_EPOLL</code>	6	epoll (stub)
<code>FD_EVENTFD</code>	7	eventfd
<code>FD_SIGNALFD</code>	8	signalfd (stub)
<code>FD_TIMERFD</code>	9	timerfd (stub)
<code>FD_PIDFD</code>	10	pidfd (stub)

表 6.2: 文件对象类型枚举

`struct file` 通过联合体 `f_data` 将不同后端指向同一入口:

```
struct file {
    enum { FD_NONE, FD_PIPE, FD_REG, FD_DEVICE, FD_BUSYBOX, ... } f_type;
    int f_flags; // O_RDONLY / O_WRONLY / O_RDWR
    int f_count; // 引用计数
    uint f_pos; // 读写偏移
    int f_major; // 设备号 (FD_DEVICE 专用)
    int removed; // 延迟删除标记
```

```

char f_path[MAXPATH];
union {
    struct pipe *f_pipe; // FD_PIPE
    file_vnode_t f_vnode; // FD_REG / FD_DEVICE
} f_data;
};

```

6.2.2 生命周期管理

函数	语义
<code>filealloc()</code>	从全局表 <code>f_table[]</code> 分配引用计数为 1 的空文件对象
<code>fdalloc(f)</code>	在进程 <code>ofile[]</code> 中找空槽，将文件对象绑定到文件描述符
<code>filedup(f)</code>	增加文件对象引用计数 (<code>f_count++</code>)，用于 <code>dup/fork</code>
<code>fileclose(f)</code>	减少引用计数；归零时调用后端关闭逻辑

`fileclose` 的关键行为是延迟删除：先持有 `f_table.lock` 递减 `f_count`；归零后将状态保存到栈上，清空原对象，再根据 `f_type` 分发到对应后端的关闭逻辑。若 `removed` 标记为真，额外调用删除操作。

6.2.3 读写类型分发

`fileread` 和 `filewrite` 按 `f_type` 分发：

普通文件与目录请求转入 EXT4 适配层，设备文件转入字符设备接口，管道转入管道子系统，BusyBox 虚拟文件则直接由内核生成内容。借助这一统一入口，系统调用层无需感知后端差异即可完成读写分发。

6.2.4 FILE_OPS 分发表

系统通过全局结构体 `FILE_OPS` 将文件操作集中管理：

```

struct file_operations FILE_OPS = {
    .dup = &filedup,
    .close = &fileclose,
    .read = &fileread,
    .readat = &filereadat,
    .write = &filewrite,
    .fstat = &filestat,
    .statx = &filestatx,
    .writable = &filewritable,
    .readable = &filereadable,
};

```

6.3 EXT4 适配层

`kernel/fs/vfs_ext4.c` 是将内核文件对象与 `lwext4` 库连接的关键适配层。它不重新实现 EXT4 语义，而是将内核侧的数据结构转换为 `lwext4` API 的参数。

6.3.1 打开路径：`vfs__ext4__openat`

`vfs_ext4_openat` 的处理流程：

1. 调用 `vfs_ext4_is_dir` 判断目标路径是否为目录（通过 `ext4_raw_inode_fill` 读取原始 inode，检查 `EXT4_INODE_MODE_DIRECTORY`）；
2. 若是目录：分配目录 vnode（`vfs_alloc_dir`），调用 `ext4_dir_open`；
3. 若是文件：分配文件 vnode（`vfs_alloc_file`），调用 `ext4_fopen2`；
4. 再次读取原始 inode，若发现字符设备类型，则将文件类型改为 `FD_DEVICE` 并记录 `f_major`；否则保持 `FD_REG`。

6.3.2 读写路径：分页中转缓冲区

`vfs_ext4_read` 和 `vfs_ext4_write` 采用分页式中转机制处理用户空间地址：

```
int vfs_ext4_read(struct file *f, int user_addr,
                  const uint64 addr, int n)
{
    char *buf = kalloc(); // 分配一页作为中转缓冲区
    for (has_read = 0; has_read < n;
        has_read += realread, uaddr += realread)
    {
        expect = min(n - has_read, PGSIZE);
        ext4_fread(ext4_f, buf, expect, &realread); // 读入内核页
        either_copyout(user_addr, uaddr, buf, realread); // 拷贝到用户空间
    }
    kfree(buf);
    f->f_pos = ext4_f->fpos;
    return byteread;
}
```

设计理由：`lwext4` 的 `ext4_fread` 接受连续内核缓冲区作为目标，而用户地址空间可能不连续。因此需要以 `PGSIZE` 为粒度，通过内核页作为中介进行逐段拷贝。

6.3.3 目录遍历

`vfs_ext4_getdents` 循环调用 `ext4_dir_entry_next` 获取下一个目录项，跳过 `.` 和 `..`，将 `lwext4` 目录项转换为 `linux_dirent64` 格式，计算对齐后的 `reclen` 确保不超出用户缓冲区，最后返回已写入的字节数。

6.3.4 元数据访问

统计信息函数 `vfs_ext4_stat`、`vfs_ext4_fstat`、`vfs_ext4_statx` 均通过 `ext4_raw_inode_fill` 读取原始 inode 和 superblock，然后逐字段填充 `struct kstat` 的模式、大小、时间戳（含纳秒部分从 `extra` 字段提取）。

6.3.5 删除操作中的 inode 引用协调

`vfs_ext4_unlinkat` 需要显式协调父目录和子节点的 inode 引用：获取挂载点，通过 `ext4_raw_inode_fill` 查找两者的 inode 号，获取 `inode_ref` 并加锁，通过 `strrchr` 提取子节点名称，调用 `_ext4_unlink` 完成底层删除，最后释放两个 `inode_ref`。这一流程说明目录项删除被视为需要在适配层明确协调的元数据操作。

6.4 块缓存层

块缓存层（`kernel/fs/bio.c`）为文件系统提供统一的磁盘缓冲抽象，维护一个双向链表按最近使用顺序排序。

6.4.1 核心数据结构

```
struct {
    struct spinlock lock; // 保护缓存元数据
    struct buf buf[NBUF]; // 缓冲块数组
    struct buf head; // 链表头节点（head.next 为最近使用）
} bcache;
```

每个 `struct buf` 包含设备号、块号、有效位、磁盘持有标记、引用计数、睡眠锁和数据区（`B_SIZE` 字节）。

6.4.2 四个核心接口

函数	行为
<code>bget(dev, blockno)</code>	查找缓存命中；未命中则回收最久未使用的空闲块
<code>bread(dev, blockno)</code>	<code>bget</code> 后检查 <code>valid</code> ；无效时调用底层驱动读入
<code>bwrite(b)</code>	将缓冲块内容写回磁盘
<code>brelse(b)</code>	释放睡眠锁，将块移到 LRU 链表头部

6.4.3 bget 的 LRU 回收策略

`bget` 从头（最近使用）向尾遍历已缓存块，若命中则增加 `refcnt`，返回锁定缓冲；若未命中，从尾（最久未使用）向前遍历，寻找 `refcnt == 0` 的空闲块，重新初始化后返回；若所有块均被引用，触发 `panic`。

```

struct buf* bread(uint dev, uint blockno) {
    b = bget(dev, blockno);
    if (!b->valid) {
#ifdef RISC_V
        virtio_rw(b, 0); // RISC-V: virtio-mmio
#else
        la_virtio_disk_rw(b, 0); // LoongArch: virtio-pci
#endif
        b->valid = 1;
    }
    return b;
}

```

6.4.4 blockdev.c 的 EXT4 适配

kernel/fs/blockdev.c 将 bio.c 的接口封装为 lwext4 需要的 ext4_blockdev 模式。读路径通过 bread 读取块后 memmove 到目标缓冲区再 brelse；写路径类似，使用 bget + memmove + bwrite + brelse 的顺序：

```

static int blockdev_read(struct ext4_blockdev *bdev,
    void *buf, uint64_t blk_id, uint32_t blk_cnt)
{
    for (int i = 0; i < blk_cnt; i++) {
        struct buf *b = bread(0, blk_id + i);
        memmove(buf + i*BSIZE, b->data, BSIZE);
        brelse(b);
    }
    return EOK;
}

```

这样，上层 EXT4 语义请求经过 EXT4 语义 -> blockdev 适配 -> 操作系统缓冲缓存 -> VirtIO 驱动四层到达物理设备。

6.5 管道实现

管道实现位于 kernel/fs/pipe.c，使用环形缓冲区和累计计数管理读写位置。

```

struct pipe {
    struct spinlock lock;
    char data[PIPESIZE]; // 512 字节固定缓冲区
    uint nread; // 累计读取字节数
    uint nwrite; // 累计写入字节数
    int readopen; // 读端是否打开
}

```

```
int writeopen; // 写端是否打开
};
```

缓冲区满条件为 `nwrite == nread + PIPESIZE`，避免了模运算和环绕处理。

写路径 (`pipewrite`)：获取管道锁后循环写入，缓冲区满时唤醒读端并在 `&nwrite` 上睡眠；若读端已关闭或进程被 `kill`，返回 `-1`。每次从用户空间拷贝 1 字节写入 `data[nwrite++ % PIPESIZE]`，完成后唤醒所有等待的读端。

读路径 (`piperead`)：获取管道锁，若无数据且写端仍打开，在 `&nread` 上睡眠。逐字节从 `data[nread++ % PIPESIZE]` 拷贝到用户空间，完成后唤醒写端。

6.6 BusyBox 虚拟文件

为支撑 BusyBox 和兼容测例的运行，系统在真实 EXT4 之外引入了一套轻量级虚拟文件机制。

6.6.1 虚拟文件列表

路径	内容类型	说明
<code>/proc/meminfo</code>	固定字符串	内存总量，分架构（RISC-V: 1GB / LA: 400MB）
<code>/proc/mounts</code>	固定字符串	两个挂载项（ <code>rootfs + proc</code> ）
<code>/proc/cpuinfo</code>	固定字符串	CPU 信息，分架构描述
<code>/proc/config.gz</code>	空字符串	兼容性占位
<code>/proc/self/maps</code>	固定字符串	用户态内存映射示意
<code>/proc/self/exe</code>	固定字符串	当前可执行文件路径
	<code>"/busybox"</code>	
<code>/proc/self/status</code>	动态生成	含 PID、PPID、UID 等
<code>/proc/<pid>/stat</code>	动态生成	按 PID 查询进程状态
<code>/proc/sys/kernel/pid_max</code>	固定字符串	<code>"32768"</code>
<code>/proc/sys/fs/pipe-user-pages-soft</code>	固定字符串	<code>"16384"</code>
<code>/dev/misc/rtc</code>	空字符串	RTC 占位
<code>/etc/passwd</code>	固定字符串	<code>root</code> 和 <code>nobody</code> 两个用户
<code>/etc/group</code>	固定字符串	<code>root</code> 和 <code>nogroup</code> 两个组
<code>/proc/self/oom_score_adj</code>	动态 OOM 值	可读可写
<code>/proc/<pid>/oom_score_adj</code>	动态 OOM 值	按 PID 读写

表 6.6: BusyBox 虚拟文件列表

6.6.2 按路径分发

`busybox_virtual_content` (`kernel/fs/file.c`) 通过字符串比较实现路径分发表。`is_busybox_virtual_path` 通过 `get_fs_from_path` 返回 `NULL` 且路径匹配虚拟文件前缀来判断。对于目录类型 (`/proc`、`/proc/self`、`/etc`)，在 `sys_openat` 中也进行特殊处理。

6.7 execve 装载器

程序装载由 `sys_execve` (`kernel/syscall.c`) 和 `exec` (`kernel/exec.c`) 两级组成。

6.7.1 执行流程

`sys_execve()` 负责从用户空间复制路径、参数和环境变量，并完成兼容层预处理；随后调用 `exec()` 重新构造当前进程的用户地址空间，装载 ELF 或脚本解释器、建立用户栈并写入辅助向量，最后将控制权切换到新的用户入口。该过程保持进程身份不变，但以新的代码段、数据段和栈布局替换原有执行映像。

6.7.2 PT_LOAD 段装载与 loadseg

`loadseg` 是 ELF 段装载的核心函数，通过 `inode` 操作接口直接读取文件内容到已分配物理页，避免经过用户态缓冲区的额外拷贝：

```
static int loadseg(pgtbl_t pt, uint64 va, struct inode *ip,
                  uint offset, uint sz)
{
    for (i = 0; i < sz; i += PGSIZE) {
        pa = walkaddr(pt, va + i);
        n = MIN(sz - i, PGSIZE);
        ip->i_op->read(ip, 0, (uint64)pa,
                      offset + i, n);
    }
    return 0;
}
```

6.7.3 PIE 支持

对于 `ET_DYN` 类型的 ELF (PIE 可执行文件)，装载器在 `load_bias = 0x10000` 的基础上映射：

```
if (ehdr.type == ELF_TYPE_DYN)
    load_bias = 0x10000UL;
uint64 seg_vaddr = load_bias + ph.vaddr;
```

选择 `0x10000` 作为基址的设计理由：保持低地址空页（`NULL` 解引用可捕获），同时使 PIE 程序的 `AT_*` 辅助向量值在动态链接器的合理范围内。

6.7.4 动态解释器处理

当 ELF 包含 `PT_INTERP` 程序头时，`exec` 进入动态链接路径。首先读取 `interp_name` 字符串（如 `/lib/ld-linux-riscv64-lp64d.so.1`），然后根据解释器名称匹配 `glibc` 或 `musl` 的对应路径，读取解释器文件的 ELF 头部，调用 `load_interpreter` 通过 `mmap` 分

配连续的虚拟地址空间，逐 PT_LOAD 段通过 loadseg 装载。

6.7.5 辅助向量构造

辅助向量（auxv）是内核向用户态动态链接器传递信息的关键机制。exec 通过 alloc_aux 构建：

辅助向量类型	值	用途
AT_HWCAP	0	硬件能力位图
AT_PAGESZ	PGSIZE (4096)	页大小
AT_PHDR	load_bias + ph.vaddr	程序头表地址
AT_PHENT	ehdr.phentsize	每个程序头大小
AT_PHNUM	ehdr.phnum	程序头数量
AT_BASE	interp_start_addr	动态链接器基址
AT_ENTRY	load_bias + ehdr.entry	程序入口点
AT_UID	0	用户 ID
AT_EUID	0	有效用户 ID
AT_GID	0	组 ID
AT_EGID	0	有效组 ID
AT_SECURE	0	安全模式标志
AT_RANDOM	栈上的 16 字节随机数地址	ASLR 熵来源
AT_FLAGS	0	标志位
AT_NULL	0	结束标记

表 6.8: 用户栈上辅助向量列表

随机数使用固定值 {0x7be6f23c6eb43a7e, 0xb78b3ea1f7c8db96}，写入栈后通过 AT_RANDOM 向动态链接器传递地址。

6.7.6 脚本兼容与重定向

.sh 脚本识别由 is_sh_script 通过文件名后缀比较完成：

```
int is_sh_script(char *path) {
    int len = strlen(path);
    if (len < 3) return 0;
    return (path[len-1]=='h' && path[len-2]=='s' && path[len-3]=='.' );
}
```

若命中，exec 将目标文件替换为 /musl/busybox，参数改写为 busybox sh [原参数]。

重定向解析在 argv 中扫描 > 和 >> 符号。当在入口切换前检测到重定向时，exec 关闭当前的标准输出并调用 vfs_ext4_open 以 O_WRONLY 或 O_WRONLY | O_APPEND 模式打

开目标文件。

6.8 本章小结

当前文件系统以 `struct file` 统一文件对象承接系统调用，以 `get_absolute_path` 和 `get_fs_from_path` 组织命名空间，以 `vfs_ext4.c` 对接 `lwext4`，以 `bio.c` 和 `blockdev.c` 贯通到底层块设备驱动。BusyBox 虚拟文件层补齐 `/proc`、`/etc`、`/dev/shm` 等运行时需求，`execve` 装载器将文件接口转化为程序加载能力。

第七章 驱动层设计与实现

7.1 概述

驱动层将硬件事件转化为内核可消费的接口。不同体系结构在寄存器布局、总线形式和中断入口上存在差异，但驱动层需要向文件系统和调度器提供稳定的抽象。当前驱动层组织为两条后端：RISC-V 侧以 VirtIO MMIO 驱动块设备，LoongArch 侧以 PCI/virtio-pci 驱动块设备；控制台、定时器和中断分发作为公共支撑层。

驱动层各模块的依赖关系如图所示：

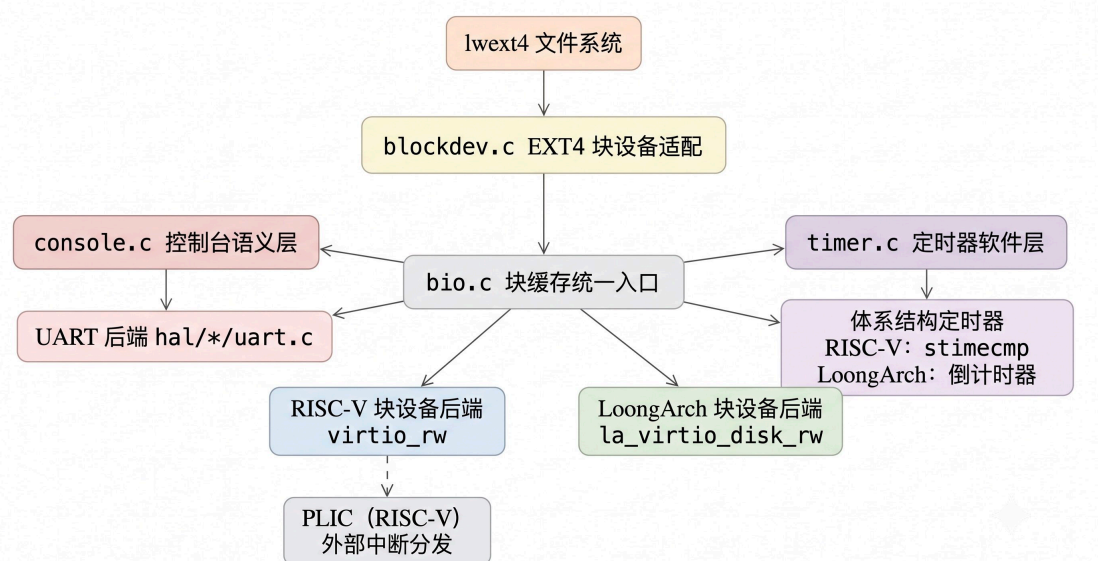


图 7.1: 驱动层模块依赖关系

7.2 控制台子系统

7.2.1 读写分离架构

控制台层（kernel/console.c）将字符输入输出的语义与硬件访问分离。核心接口分为三条路径：

7.2.2 行规约处理

consoleintr 是控制台输入的统一入口，在持有 cons.lock 的条件下处理特殊输入：Ctrl-P (0x10) 触发 procdump()，Ctrl-U (0x15) 清除整行，退格 (Ctrl-H) 删除最后一个字符，回车 (r) 规整为 n，Ctrl-D (0x04) 触发文件结束语义；普通字符写入编辑区，遇到换行或缓冲填满时推进写入边界并唤醒阻塞的读者。

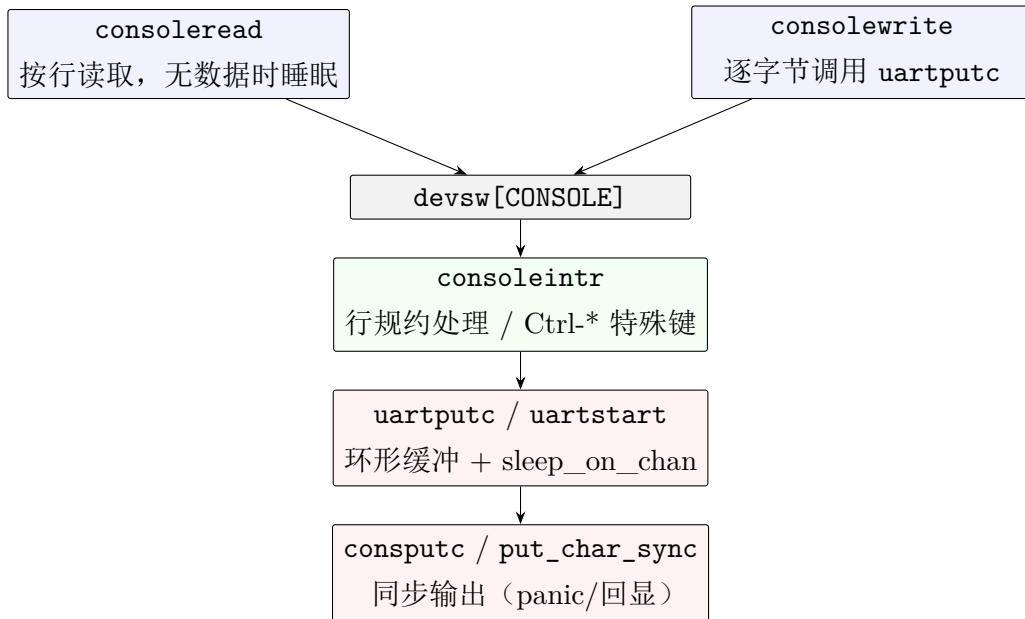


图 7.2: 控制台的读写分离架构

控制台输入缓冲区用三个指针维护：

```

struct {
    struct spinlock lock;
    char buf[INPUT_BUF_SIZE]; // 128 字节
    uint r; // 读取位置（已提交给读者的数据）
    uint w; // 写入边界（已确认的可读数据）
    uint e; // 编辑位置（当前行编辑位置）
} cons;
  
```

7.2.3 UART 发送环形缓冲

uartputc 和 uartstart (hal/*/uart.c) 构成写路径的协作对。uartputc 在环形缓冲满时通过 sleep_on_chan(&uart_tx_r) 睡眠；uartstart 从缓冲区取出字符写入 UART 数据寄存器，缓冲变空时唤醒等待的写者。put_char_sync 用于回显和 panic 等同步输出场景，轮询 UART 线路状态直到发送器空闲。

7.2.4 字符设备注册

chardev_init 完成控制台设备节点注册：初始化 cons.lock，调用 uart_init() 初始化 UART 硬件，将 consoleread 和 consolewrite 绑定到 devsw[CONSOLE]，注册 /dev/null 和 /dev/zero。

7.3 定时器

7.3.1 双架构定时器初始化对比

RISC-V 和 LoongArch 采用不同的硬件定时器机制：

特性	RISC-V	LoongArch
定时器类型	比较寄存器 <code>stimecmp</code>	倒计时计数器 (countdown)
中断触发条件	<code>time >= stimecmp</code>	计数减到 0
重装方式	每次中断后软件写 <code>stimecmp</code>	硬件自动重装 (periodic 位)
SBI 路径	<code>set_timer(r_time() + INTERVAL)</code>	不支持
非 SBI 路径	<code>w_stimecmp(r_time() + INTERVAL)</code>	<code>w_csr_tcfg(TCFG)</code>

表 7.2: RISC-V 与 LoongArch 定时器机制对比

RISC-V 的 `timer_init` (`kernel/timer.c`) 初始化 `tickslock`、清零 `ticks`，并通过 `read_rtc_seconds()` 读取 RTC 寄存器获得 `boot_time`。LoongArch 的 `countdown_timer_init` 配置 TCFG 寄存器使能周期性倒计时：

```
void countdown_timer_init(void)
{
    prcfg1_val = r_csr_prcfg1();
    timerbits = FIELD_GET(prcfg1_val, TIMERBITS) + 1;
    // 构造 TCFG 值：使能 + 周期重装 + 初始值
    tcfg_val = FIELD_WRITE(1, TCFG_EN) |
                FIELD_WRITE(1, TCFG_PERIODIC) |
                FIELD_WRITE(INTERVAL, TCFG_INITVAL(timerbits));
    w_csr_tcfg(tcfg_val);
}
```

7.3.2 事件链：timer_tick 到 SIGALRM

`timer_tick` 是定时器中断的公共处理逻辑：递增全局 `ticks`，唤醒等待时钟推进的睡眠者；遍历进程池对每个进程调用 `refresh_process_timer`；RISC-V 上预约下一次中断，LoongArch 上倒计时器自动重装。

`refresh_process_timer` 将进程级 interval timer 绑定到时间轴：

```
static void refresh_process_timer(proc_t *p, uint64 now)
{
    if (!p->timer_active || now < p->alarm_ticks)
        return;
    // 超时：设置 SIGALRM 信号
    p->sig_pending.__val[0] |= (1UL << SIGALRM);
}
```

```

interval = timeval_to_clocks(&p->itimer.it_interval);
if (interval == 0) {
    // 一次性定时器
    p->timer_active = 0;
} else {
    // 周期性定时器: 推进 alarm_ticks
    do {
        p->alarm_ticks += interval;
    } while (p->alarm_ticks <= now);
}
}

```

7.4 RISC-V PLIC

平台级中断控制器 (PLIC) 用于管理和分发 RISC-V 的外部中断。实现位于 `hsai/plic.c`。 `plicinit()` 为 UART0 和 VIRTIO0 设置优先级, `plicinithart()` 使能 S 态中断源, `plic_claim()` 获取待处理中断 IRQ, `plic_complete()` 通知 PLIC 处理完成。

在 Trap 层 (`hsai/hsai_trap.c`) 的 `devintr()` 中, PLIC 用于设备中断分类:

```

int devintr() {
    // RISC-V 路径
    scause = r_scause();
    if (scause == SUPERVISOR_EXTERNAL_INTERRUPT) {
        irq = plic_claim();
        if (irq == VIRTIO0_IRQ)
            virtio_disk_intr(); // 回收块设备请求
        else if (irq == UART0_IRQ)
            uartintr(); // 处理串口输入
        plic_complete(irq);
        return 1;
    } else if (scause == SUPERVISOR_TIMER_INTERRUPT) {
        timer_tick();
        return 2;
    }
    return 0;
}

```

7.5 RISC-V VirtIO MMIO 块设备

RISC-V 块设备驱动位于 `kernel/driver/riscv/virt.c`, 采用 MMIO 方式与 VirtIO 设备通信。

7.5.1 virtio_disk_init 状态机

初始化遵循 VirtIO 协议的标准状态机：

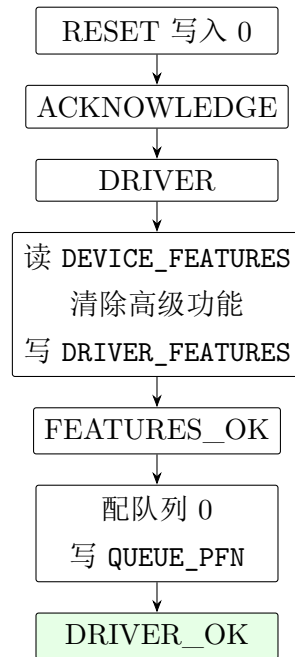


图 7.3: virtio_disk_init 的状态机

特性协商中显式关闭的能力包括：VIRTIO_BLK_F_RO（只读）、VIRTIO_BLK_F SCSI、VIRTIO_BLK_F_CONFIG_WCE（写缓存）、VIRTIO_BLK_F_MQ（多队列）、VIRTIO_F_ANY_LAYOUT、VIRTIO_RING_F_EVENT_IDX 和 VIRTIO_RING_F_INDIRECT_DESC。这表示当前实现选择了一条保守但容易验证的 VirtIO 块设备子集。

7.5.2 三描述符链与请求提交

virtio_rw 对每次块请求构造三个描述符：

请求提交后通过忙等待步：

```

b->disk = 1; // 标记 buf 为设备持有
disk.avail->ring[disk.avail->idx % NUM] = idx[0];
__sync_synchronize();
disk.avail->idx += 1; // 通知设备有可用描述符
__sync_synchronize();
*R(VIRTIO_MMIO_QUEUE_NOTIFY) = 0; // 触发设备处理

while (b->disk == 1) { } // 忙等请求完成
  
```

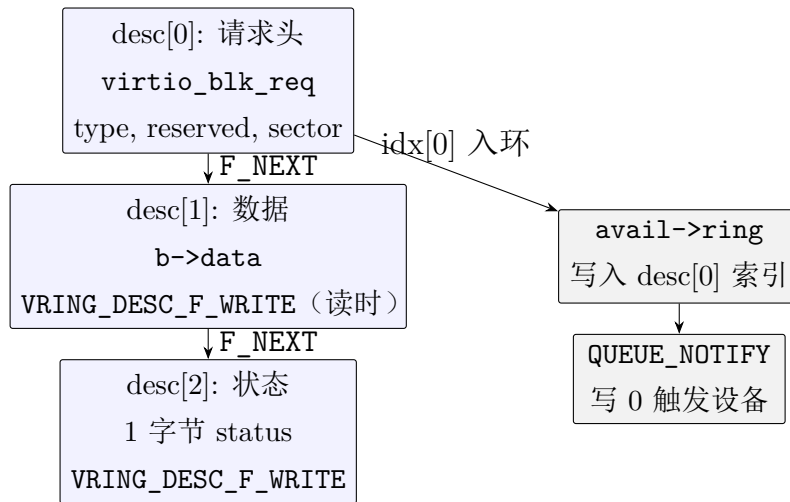


图 7.4: virtio_rw 的三描述符链

7.5.3 中断回收

virtio_disk_intr 被 Trap 层的 devintr 在识别到 VIRTIO0_IRQ 后调用：读 INTERRUPT_STATUS 并写入 INTERRUPT_ACK 确认中断，然后循环处理 used 环中所有新完成项，通过 info[id].b 获取对应的 struct buf 并将 b->disk 清零。

7.5.4 描述符表内存布局

```
// pages 是两页对齐的内存
disk.desc = (struct virtq_desc *)pages; // 描述符表
disk.avail = (struct virtq_avail *) (pages + // 可用环
    NUM * sizeof(struct virtq_desc));
disk.used = (struct virtq_used *) (pages + PGSIZE); // 已用环
```

三个队列在共享内存中的布局以页边界分割：第一页包含描述符表和可用环，第二页包含已用环。设备通过 QUEUE_PFN 寄存器和页框号访问这片共享内存。

7.6 LoongArch PCI 与 VirtIO-PCI 块设备

LoongArch 块设备后端由三个文件构成：kernel/driver/loongarch/pci.c、kernel/driver/loongarch/virtio_pci.c 和 kernel/driver/loongarch/virtio_disk.c。

7.6.1 PCI 设备枚举

pci_device_probe 在 ECAM (Enhanced Configuration Access Mechanism) 空间中遍历 bus、device 和 function:

```
uint64 pci_device_probe(uint16 vendor_id, uint16 device_id)
{
    for (bus = 0; bus < 256; bus++)
        for (dev = 0; dev < 32; dev++)
            for (func = 0; func < 8; func++) {
```

```

        uint64 off = (bus << 16) | (dev << 11) | (func << 8);
        uint32 id = pci_config_read32(off + PCI_VENDOR_ID);
        if (id == (vendor_id | (device_id << 16))) {
            // 匹配成功: 记录 bus/dev/func
            // 写 PCI_COMMAND 开启 I/O、memory、bus master
            return off; // 返回 ECAM 偏移
        }
    }
}

```

7.6.2 Capability 解析

virtio_pci_read_caps 从配置空间偏移 0x34 读取 capability 链表，遍历所有 vendor-specific capability:

```

int virtio_pci_read_caps(virtio_pci_hw_t *hw, uint64 pci_base,
                        trap_handler_fn *msix_isr)
{
    cap_ptr = pci_config_read8(pci_base + PCI_CAPABILITY_LIST);
    while (cap_ptr) {
        cap_id = pci_config_read8(pci_base + cap_ptr);
        cap_len = pci_config_read8(pci_base + cap_ptr + 1);
        if (cap_id == PCI_CAP_ID_VENDOR) {
            cfg_type = pci_config_read8(pci_base + cap_ptr + 3);
            bar = pci_config_read8(pci_base + cap_ptr + 4);
            offset = pci_config_read32(pci_base + cap_ptr + 8);
            // 根据 cfg_type 分别保存:
            // common_cfg, notify_cfg, device_cfg, isr_cfg
        }
        cap_ptr = pci_config_read8(pci_base + cap_ptr + 2);
    }
    // 四类能力缺一即判定为非现代 virtio-pci 设备
}

```

7.6.3 轮询完成路径

la_virtio_disk_rw (kernel/driver/loongarch/virtio_disk.c) 与 RISC-V 版本有三点关键差异:

1. **锁保护:** 获取 disk.vdisk_lock 防止重入，因为 LoongArch 路径不使用独立中断回调;
2. **地址转换:** 描述符地址通过 PA2VA() 去除直接映射高位 (DMWIN_MASK)，使队列项中的地址符合设备可见的物理布局;

3. 完成同步：不依赖中断回调，直接在提交后轮询 used 环：

```
acquire(&disk.vdisk_lock);
// ... 构造三描述符链 ...
disk.avail[2 + (disk.avail[1] % NUM)] = idx[0];
__sync_synchronize();
disk.avail[1] = disk.avail[1] + 1;
virtio_pci_set_queue_notify(&gs_virtio_blk_hw, 0);

while (disk.used_idx == disk.used->id) { } // 轮询完成
int id = disk.used->elems[disk.used_idx % NUM].id;
// 检查状态字
disk.info[id].b->disk = 0;
disk.used_idx += 1;
free_chain(id);
release(&disk.vdisk_lock);
```

7.6.4 两种架构的差异对比

特性	RISC-V VirtIO MMIO	LoongArch VirtIO-PCI
设备接口	MMIO 寄存器	PCI 配置空间 + BAR
队列通信	共享内存 (QUEUE_PFN)	set_queue_addr2
通知方式	写 QUEUE_NOTIFY 寄存器	set_queue_notify
完成回收	独立中断回调 virtio_disk_intr	内部轮询 used->id
请求同步	while(b->disk==1) 忙等	while(used_idx==used->id) 忙等
并发控制	无锁 (单线程假设)	vdisk_lock 互斥锁
特性协商	MMIO 状态寄存器	PCI 能力寄存器

表 7.4: RISC-V 与 LoongArch 块设备驱动对比

7.7 驱动与上层接口

kernel/fs/bio.c 是驱动层与文件系统的衔接面。bread 和 bwrite 通过条件编译分发到对应后端，对上层表现为 1024 字节块设备。

kernel/fs/blockdev.c 将 bio.c 接口封装为 lwext4 所需的 ext4_blockdev 接口，填充 bread、bwrite、open、close 等回调函数。blockdev_read 以顺序块循环方式逐一读写：

```
static int blockdev_read(struct ext4_blockdev *bdev,
```

```
void *buf, uint64_t blk_id, uint32_t blk_cnt)
{
    for (int i = 0; i < blk_cnt; i++) {
        struct buf *b = bread(0, blk_id + i);
        memmove(buf + i*BSIZE, b->data, BSIZE);
        brelse(b);
    }
    return EOK;
}
```

I/O 主路径为：lwext4 调用块设备读写接口，进入 bio 层，下沉到 VirtIO 驱动，最后访问物理设备。

7.8 本章小结

驱动层包含三个功能方向：字符设备以 `kernel/console.c` 为语义中心，以 `hal/*/uart.c` 为硬件后端，通过 `devsw[]` 接入 VFS；定时器通过 `kernel/timer.c` 将硬件时钟中断转化为进程定时器和 SIGALRM 信号，经 `yield()` 与调度器衔接；块设备在 RISC-V 上采用 `virtio-mmio + PLIC` 中断回收，在 LoongArch 上采用 `PCI/virtio-pci + 同步轮询`，统一汇入 `kernel/fs/bio.c` 的 `bread/bwrite` 接口。

第八章 总结与展望

8.1 设计总结

各章核心设计决策如下：

- 第 2 章（进程与线程）：以 `proc_t` 持有全部资源（页表、VMA、文件描述符、信号状态），以 `thread_t` 持有执行现场（`trapframe`、`context`、内核栈），以 `cpu_t` 持有调度器上下文；`scheduler()` 在进程可运行的前提下选择线程切换。
- 第 3 章（内存管理）：通过 Buddy 分配物理页、Slab 管理小对象、页表控制硬件映射、VMA 描述虚拟区间，四层构成从物理内存到用户态缺页处理的完整链路。
- 第 4 章（HAL 与 Trap 抽象）：`hal/` 提供架构入口和现场操作汇编例程；`hsai/` 将 Trap 分发、页表配置和中断控制封装为统一接口，使 `usertrap()`、`kerneltrap()` 和 `pagefault_handler()` 可在两个架构上共享函数体。
- 第 5 章（系统调用与用户态接口）：`kernel/syscall.c` 的分发表覆盖 Linux 兼容的系统调用类别；`exec()` 完成脚本检测、ELF/动态链接装载和环境注入；用户态通过 `ecall`（RISC-V）或 `syscall 0`（LoongArch）陷入后统一汇入 `usertrap()`。
- 第 6 章（文件系统与程序装载）：集成 `lwext4` 库并在其上构建路径解析、文件描述符、标准设备节点、虚拟路径（`/proc`、`/dev/shm`）和块缓存层；`kernel/exec.c` 在此基础上完成 ELF 和脚本装载。
- 第 7 章（驱动层）：RISC-V 使用 VirtIO MMIO，LoongArch 通过 PCI 枚举发现 VirtIO-PCI 设备；两者通过 `lwext4` 块设备接口向文件系统提供统一的块 I/O。

8.2 后续优化方向

表 8.1: 后续优化方向

方向	当前状态	计划改进
调度与信号语义	<code>scheduler()</code> 、 <code>futex_wait()</code> 、 <code>timer_tick()</code> 和信号挂起机制已能驱动运行期控制流	补充线程定向信号、 <code>signal frame</code> 递送、 <code>clear_child_tid</code> 和线程退出语义闭合
内存权限与并发	Buddy、页表、VMA 和共享映射已形成层次基础	完善 TLB 同步、权限细化（VMA 权限与 PTE 权限对齐）、共享页一致性和多路径并发保护
文件系统标准化	EXT4 主路径和虚拟文件兼容已支撑 BusyBox/LTP 等场景	完善挂载语义、路径规则一致性、缓存热路径和运行时兼容细节（ <code>inotify</code> 、 <code>epoll</code> 等）

驱动与性能	RISC-V VirtIO MMIO 和 LoongArch PCI/VirtIO-PCI 已接 通块 I/O 通路	优化中断完成 path、中断驱动细 节、新设备类型接入和 I/O 性能 稳定性
-------	--	---
